

外发版-评测题包交付规范 (rewardkit) 20260913

注意：需要和“外发版-基于weakness和skill 的数据构造方案”组合使用

外发版-评测题包交付规范（rewardkit）

更新时间：20260913

面向供应商。本文档规定题包格式与评分器写法；题量配比、难度、通过率等验收标准见第7章。第6章的 `test.sh` / `finalize.py` 是平台固定模板，见附录A，逐字复制、不得改动。

1. 速览

1.1 一题一目录

目录名 = 题目编号。

FIN1-SKLII-DEP-001/ # 目录名

- |—— instruction.md # 任务描述（仅接受此文件名）
- |—— task.toml # 配置 + 元数据
- |—— rubrics.json # 原始评分细则
- |—— environment/ # 执行环境（构建上下文 = 本目录）
 - | |—— Dockerfile # 单文件含全部依赖
 - | |—— requirements.txt # 无依赖交空文件
 - | |—— skills/ # 有 Skill 时提供，与 metadata.skill_set 对应
 - | |—— <skill_name>/
 - | |—— SKILL.md # 技能说明及 Workflow/SOP
 - | |—— scripts/ # 按需提供操作脚本
 - | |—— references/ # 按需提供引用资料
 - | |—— input_files/ # 源文件 → 容器 /app/input_files
- |—— 2023年报.pdf
- |—— 2024年报.pdf
- |—— 可比公司估值数据.csv
- |—— 交易流水.csv
- |—— solution/ # 参考答案
- |—— solve.sh # Oracle 入口

- | └── golden_output/
- | └── FIN-T2-001_财务分析报告.xlsx
- | └── FIN-T2-001_毛利率趋势.png
- | └── tests/ # 评分器
 - | └── test.sh # 固定模板, 见附录 A.1
 - | └── finalize.py # 固定模板, 见附录 A.2
 - | └── rubrics.toml # 计分 Rubric
 - | └── prompt.md # 评分 agent 提示词, 见 6.2
 - | └── __golden_output/ # 参考答案
 - | └── __assets/ # 可选: 基准表等评分材料



复制代码

```

FIN1-SKLII-DEP-001/                                # 目录名
├── instruction.md                                    # 任务描述 (仅接受此文件名)
├── task.toml                                         # 配置 + 元数据
├── rubrics.json                                     # 原始评分细则
├── environment/                                     # 执行环境 (构建上下文 = 本目录)
│   ├── Dockerfile                                  # 单文件含全部依赖
│   ├── requirements.txt                             # 无依赖交空文件
│   └── skills/                                       # 有 Skill 时提供, 与
metadata.skill_set 对应
│   ├── <skill_name>/
│   │   ├── SKILL.md                                # 技能说明及 Workflow/SOP
│   │   ├── scripts/                                # 按需提供操作脚本
│   │   └── references/                              # 按需提供引用资料
│   └── input_files/                                # 源文件 → 容器 /app/input_files
│       ├── 2023年报.pdf
│       ├── 2024年报.pdf
│       ├── 可比公司估值数据.csv
│       └── 交易流水.csv
├── solution/                                        # 参考答案
│   ├── solve.sh                                    # Oracle 入口
│   └── golden_output/
│       ├── FIN-T2-001_财务分析报告.xlsx
│       └── FIN-T2-001_毛利率趋势.png
└── tests/                                           # 评分器
    ├── test.sh                                     # 固定模板, 见附录 A.1
    ├── finalize.py                                 # 固定模板, 见附录 A.2
    ├── rubrics.toml                                # 计分 Rubric
    ├── prompt.md                                   # 评分 agent 提示词, 见 6.2
    ├── __golden_output/                            # 参考答案
    └── __assets/                                    # 可选: 基准表等评分材料
  
```

1.2 组件清单

组件	必交	大小	作用
<code>instruction.md</code>	是	$\leq 1 \text{ MiB}$	任务书，作为 prompt 交给 Agent
<code>task.toml</code>	是	$\leq 1 \text{ MiB}$	Harbor 配置 + 元数据 + 交付物清单 + Rubric 索引
<code>rubrics.json</code>	是	—	原始评分细则
<code>environment/Dockerfile</code>	是	—	任务镜像层，含全部依赖
<code>environment/requirements.txt</code>	是	—	执行侧依赖，构建期预装；无依赖交空文件
<code>environment/input_files/</code>	是	计入整批 $\leq 20 \text{ GB}$	全部输入材料
<code>environment/skills/</code>	有 Skill 时必交	计入整批 $\leq 20 \text{ GB}$	<code>skill_set</code> 对应的技能目录，含 <code>SKILL.md</code> 及所需脚本、引用资料；Workflow 可由技能内 SOP 承载
<code>solution/solve.sh</code>	是	—	Oracle 入口（正向预检基准）
<code>solution/golden_output/</code>	是	计入整批 $\leq 20 \text{ GB}$	专家标准答案
<code>tests/test.sh</code>	是	—	固定模板
<code>tests/finalize.py</code>	是	—	固定模板
<code>tests/rubrics.toml</code>	是	$\leq 2 \text{ MiB}$	由 rubric 转换的计分 criteria

tests/prompt.md	是	—	评分 agent 提示词，须含 {criteria} 占位符（见 6.2）
tests/__golden_output/	是	计入整批 ≤ 20 GB	参考答案副本，供判官对照
tests/__assets/	否	计入整批 ≤ 20 GB	评分侧基准材料

1.3 容器内路径与可见性

题包内容	容器内位置	Agent 可见
instruction.md	作为 prompt	可见
environment/input_files/	/app/input_files/（只读）	可见
environment/skills/	由 Dockerfile 指定并在任务书列明	可见
environment/requirements.txt	构建期预装进镜像	可见
Agent 交付目录	/app/output/（唯一交付目录，工作目录 = /app）	可见（可写）
Agent 日志	/logs/agent/（不作为交付物）	可见
solution/	/solution（仅 Oracle 阶段）	不可见
tests/	/tests（仅评分阶段上传）	不可见
task.toml	文件不入容器	仅 [environment].env 的值可见

评分结果	<code>/logs/verifier/reward.json</code> + <code>reward.txt</code> (单一数值) + <code>reward-details.json</code> (评分明细)；评分不可用时另有 <code>reward_exit_message.json</code> (AP 错误码)	—
------	--	---

1.4 资源默认上限

项	默认值	说明
<code>[agent].timeout_sec</code>	72000 (1200 min)	A3长程任务须按实际需要上调
<code>[verifier].timeout_sec</code>	18000	须大于单个 judge 会话超时
<code>cpus</code> / <code>memory_mb</code> / <code>storage_mb</code>	2 / 8192 / 30720	可按需调整
Agent 交付物总量	≤ 2 GB	指 <code>/app/output/</code>
整批 zip	≤ 20 GB	含 inputfiles 与 goldenoutput

2. instruction.md

任务提示词，Harbor 不提供本文件的模板，由供应商自行撰写，但必须限制源文件路径为 `/app/input_files/`，交付文件路径为 `/app/output/`。

2.1 要求

- **自包含：**Agent 只能看到 `instruction.md`、`/app/input_files/` 与镜像内预装的工具、技能，任务书必须写全完成任务所需的信息，或给出镜像内可读取的技能及资料入口，不得引用外部链接或"见附件说明"之外的隐藏语境。
- **技能入口（有 Skill 时）：**列出全部可用技能的名称、适用描述及容器内 `SKILL.md` 的绝对路径，并说明脚本相对路径的基准目录；不得只写技能名称让 Agent 猜测路径。技能方法和 Workflow/SOP 保留在技能文档中，无需全文复制到任务书，也不要求特定 harness 的原生技能加载方式。不得将 `expected_skill_dependencies` 中的必要技能或干扰项身份作为发现提示泄露给 Agent。

- **列明源文件：**逐个列出 `/app/input_files/` 下的文件及其字段/口径说明（见 2.2 示例的表格写法）。
- **交付物精确命名：**写明交付物必须写入 `/app/output/`，逐个给出精确文件名——与 `[[metadata.deliverables]]`、`artifacts`、两份参考答案、`criterion description` 逐字节一致（见 3.3）。
- **与 Rubric 对齐：**Rubric 的每条显式要求都应能在任务书中找到出处；任务书中的硬性要求也应有对应 `criterion` 覆盖。
- **不得泄露评分信息：**不出现 rubric 条目、golden 内容或"评分将检查....."式提示。

2.2 参考示例

A 公司再融资定增：财务分析与估值报告

业务场景与任务说明

你是某券商投行部 ECM（股权资本市场）团队的分析师。团队正在为客户 A 公司筹备再融资定向增发，需要一份可直接进入内核会的财务分析与估值报告。报告读者是不熟悉原始报表的内核委员，请独立完成。

可使用的源文件

源文件位于 `/app/input_files/`，**只读**，不要修改：

文件	说明
2023年报.pdf	A 公司 2023 年度报告，含 2022、2023 两期合并利润表与合并资产负债表
2024年报.pdf	A 公司 2024 年度报告，含 2023、2024 两期合并利润表与合并资产负债表
交易流水.csv	A 公司 2024 年全年销售流水，UTF-8，含表头。字段依次为`流水号`、`交易日期`、`客户名称`、`客户手机号`、`客户证件号`、`产品线`、`币种`、`金额`、`是否冲销`。`是否冲销`取值为`是`/`否`，`金额`单位为元
可比公司估值数据.csv	同行业上市公司的市值、净债务、EBITDA 与营收，UTF-8，含表头

两份年报的 2023 年数据存在披露口径差异，一律以`2024年报.pdf`中追溯调整后的数据为准。

非人民币流水按`2024年报.pdf`披露的期末汇率折算为人民币。

交付物要求

交付物必须写入 `/app/output/`，不要把产物写到任何其他绝对路径。

交付物	是否必交	说明
`FIN-T2-001\_财务分析报告.xlsx`	必交	财务分析与估值报告，单个 Excel 工作簿
`FIN-T2-001\_毛利率趋势.png`	必交	2022–2024 三年毛利率折线图。横轴为年度（2022、2023、2024），纵轴为毛利率，三个数据点旁各标出该年毛利率数值

工作簿必须包含以下五个工作表，**sheet 名逐字一致**：

sheet	内容要求
`摘要`	核心结论：营收、毛利率、企业价值三项关键数字，以及一句话定增建议
`汇总表`	按月汇总 2024 年销售流水，**覆盖全部 12 个月**，中间不留空行
`毛利率`	2022–2024 三年毛利率，格式见下
`估值表`	用可比公司法给出 A 公司的企业价值（EV）。须列出至少 3 家可比公司并说明选取依据，并写明所用倍数与全部假设参数
`风险提示`	逐条列出经营风险，每条附量化依据

`毛利率` 工作表的固定格式：第 1–5 行为报表抬头（公司名、报告期、金额单位等），第 6 行为表头（`年度` / `毛利率` / `同比变动`），第 7 行为 `近三年平均`，第 8–10 行依次为 2022、2023、2024 各年。毛利率一律以小数填写（`0.5` 表示 50%），`同比变动` 列填 `上升` / `下降` / `持平`。

`风险提示` 工作表必须覆盖以下四类风险，每类一行：客户集中度、应收账款账龄、汇率敞口、原材料价格。



复制代码

A 公司再融资定增：财务分析与估值报告

业务场景与任务说明

你是某券商投行部 ECM（股权资本市场）团队的分析师。团队正在为客户 A 公司筹备再融资定向增发，需要一份可直接进入内核会的财务分析与估值报告。报告读者是不熟悉原始报表的内核委员，请独立完成。

可使用的源文件

源文件位于 `/app/input\_files/`，**只读**，不要修改：

文件	说明
`2023年报.pdf`	A 公司 2023 年度报告，含 2022、2023 两期合并利润表与合并资产负债表

| `2024年报.pdf` | A 公司 2024 年度报告，含 2023、2024 两期合并利润表与合并资产负债表 |

| `交易流水.csv` | A 公司 2024 年全年销售流水，UTF-8，含表头。字段依次为 `流水号`、`交易日期`、`客户名称`、`客户手机号`、`客户证件号`、`产品线`、`币种`、`金额`、`是否冲销`。`是否冲销` 取值为 `是` / `否`，`金额` 单位为元 |

| `可比公司估值数据.csv` | 同行业上市公司的市值、净债务、EBITDA 与营收，UTF-8，含表头 |

两份年报的 2023 年数据存在披露口径差异，一律以 `2024年报.pdf` 中追溯调整后的数据为准。

非人民币流水按 `2024年报.pdf` 披露的期末汇率折算为人民币。

交付物要求

交付物必须写入 `/app/output/`，不要把产物写到任何其他绝对路径。

| 交付物 | 是否必交 | 说明 |

| --- | --- | --- |

| `FIN-T2-001\_财务分析报告.xlsx` | 必交 | 财务分析与估值报告，单个 Excel 工作簿 |

| `FIN-T2-001\_毛利率趋势.png` | 必交 | 2022-2024 三年毛利率折线图。横轴为年度（2022、2023、2024），纵轴为毛利率，三个数据点旁各标出该年毛利率数值 |

工作簿必须包含以下五个工作表，**sheet 名逐字一致**：

| sheet | 内容要求 |

| --- | --- |

| `摘要` | 核心结论：营收、毛利率、企业价值三项关键数字，以及一句话定增建议 |

| `汇总表` | 按月汇总 2024 年销售流水，**覆盖全部 12 个月**，中间不留空行 |

| `毛利率` | 2022-2024 三年毛利率，格式见下 |

| `估值表` | 用可比公司法给出 A 公司的企业价值（EV）。须列出至少 3 家可比公司并说明选取依据，并写明所用倍数与全部假设参数 |

| `风险提示` | 逐条列出经营风险，每条附量化依据 |

`毛利率` 工作表的固定格式：第 1-5 行为报表抬头（公司名、报告期、金额单位等），第 6 行为表头（`年度` / `毛利率` / `同比变动`），第 7 行为 `近三年平均`，第 8-10 行依次为 2022、2023、2024 各年。毛利率一律以小数填写（`0.5` 表示 50%），`同比变动` 列填 `上升` / `下降` / `持平`。

`风险提示` 工作表必须覆盖以下四类风险，每类一行：客户集中度、应收账款账龄、汇率敞口、原材料价格。

3. task.toml

`environment_template`、`tool_set`、`skill_set`、`expected_tool_dependencies`、`expected_skill_dependencies` 均填写在已有 `[metadata]` 中，类型及约束见 3.2。它们声明环境能力和预期依赖，不自动安装或加载技能；实际资源及使用入口见 3.4。

3.1 参考示例

```
schema_version = "1.4" # 照抄

# 格式：/app/output/{你的交付物名称}
artifacts = [
    "/app/output/FIN-T2-001_财务分析报告.xlsx",
    "/app/output/FIN-T2-001_毛利率趋势.png",
    "/logs/artifacts/output",
]

[task]
name = "work/fin1-001" # 任务名称
version = "1.0.0" # 任务版本
description = "基于两期年报与交易流水产出财务分析报告" # 任务描述

[metadata]
task_id = "FIN1-skill-DEP-001" # 题目编号
author_organization = "<供应商名>"
category = "skill-dependency"
domain = "金融" # 所属领域：通用办公/金融/医疗/法律
domain_l2 = "投资银行" # 所属领域二级标签（对齐知识体系表）
capabilities = "财务比率计算、估值口径" # 评测的能力：该任务能评测的具体专业能力
difficulty = "A2" # 难度等级：A1易/A2中/A3难
vl_dependency = "否" # 是否依赖VL（视觉-语言能力）：是/否
source_note = "任务来源、真实性保障、真实场景中的交付物是什么" # 任务说明
tools = "Excel/Python" # 所需工具（办公工具/数据库，Word/Excel/PPT 等）
domain_knowledge = "再融资定增的财务分析口径与可比公司估值方法" # 领域知识依赖（选填）
task_complexity = "C2" # 取值C1-C5
weakness_tag = ["W07-流程跳步", "W12-约束遵循"] # 覆盖≥1个，按算法词表填写，至少一项
environment_template = "<平台已确认的环境模板名>"
tool_set = ["filesystem", "shell", "python"] # 按实际可用能力填写
skill_set = [] # 本示例未提供技能；有技能时填写目录名
expected_tool_dependencies = ["filesystem", "shell", "python"] # 按实际必要能力填写
```

```

expected_skill_dependencies = []    # 必须是 skill_set 的子集
tags = ["finance", "office", "A2", "skill-dependency", "sop-enforcement", "workflow",
"docx", "llm-judge"]
# 逐题填写任务关键词

[[metadata.deliverables]]  # 交付物一
path = "FIN-T2-001_财务分析报告.xlsx"    # 交付物一的路径
required = true
desc = "主交付物：财务分析报告"    # 交付物一的描述

[[metadata.deliverables]] # 见交付物一的说明，如有多个交付物，必须依次写明
path = "FIN-T2-001_毛利率趋势.png"
required = true
desc = "毛利率趋势折线图"

[agent]
timeout_sec = 72000.0    # 缺省 72000；

[verifier]
timeout_sec = 18000.0
user = "root"

[verifier.env]
# Judge 相关变量统一 JUDGE_ 前缀，只写在 [verifier.env]、严禁写入
[environment.env]。
# 必选项（运行环境注入，AP 必给）；可选项一律用 ${VAR:-默认值} 语法给默认值。
JUDGE_API_KEY = "${JUDGE_API_KEY:-}"    # 必选：judge 网关 key
JUDGE_BASE_URL = "${JUDGE_BASE_URL:-}"    # 必选：judge 网关地址（可带
/v1，test.sh 负责剥）
JUDGE_MODEL = "${JUDGE_MODEL:-qwen3.7-plus}"    # 可选：裁判模型（经
REWARDKIT_MODEL 运行时生效）
JUDGE_PROVIDER = "${JUDGE_PROVIDER:-anthropic}" # 可选：信息性声明
JUDGE_API_PROTOCOL = "${JUDGE_API_PROTOCOL:-anthropic}" # 可选：openai 时
降级为 LLM judge（见 test.sh）
# 兼容旧环境的兜底注入
EVAL_API_KEY = "${EVAL_API_KEY:-}"
EVAL_API_BASE = "${EVAL_API_BASE:-}"
LITELLM_DROP_PARAMS = "true"

[environment]
os = "linux"
build_timeout_sec = 18000.0

```

```
network_mode = "public"    # claude-code 框架下不要写 no-network（见 3.2 警示）
cpus = 2
memory_mb = 8192
storage_mb = 30720        # 照抄
```

[复制代码](#)

```
schema_version = "1.4"        # 照抄

# 格式: /app/output/{你的交付物名称}
artifacts = [
    "/app/output/FIN-T2-001_财务分析报告.xlsx",
    "/app/output/FIN-T2-001_毛利率趋势.png",
    "/logs/artifacts/output",
]

[task]
name = "work/fin1-001"        # 任务名称
version = "1.0.0"            # 任务版本
description = "基于两期年报与交易流水产出财务分析报告" # 任务描述

[metadata]
task_id = "FIN1-skill-DEP-001" # 题目编号
author_organization = "<供应商名>"
category = "skill-dependency"
domain = "金融"                # 所属领域: 通用办公/金融/医疗/法律
domain_l2 = "投资银行"         # 所属领域二级标签 (对齐知识体系表)
capabilities = "财务比率计算、估值口径" # 评测的能力: 该任务能评测的具体专业能力
difficulty = "A2"              # 难度等级: A1易/A2中/A3难
vl_dependency = "否"           # 是否依赖VL (视觉-语言能力): 是/否
source_note = "任务来源、真实性保障、真实场景中的交付物是什么" # 任务说明
tools = "Excel/Python"        # 所需工具 (办公工具/数据库, Word/Excel/PPT 等)
domain_knowledge = "再融资定增的财务分析口径与可比公司估值方法" # 领域知识依赖 (选填)
task_complexity = "C2"         # 取值C1-C5
weakness_tag = ["W07-流程跳步", "W12-约束遵循"] # 覆盖≥1个, 按算法词表填写, 至少一项
environment_template = "<平台已确认的环境模板名>"
tool_set = ["filesystem", "shell", "python"] # 按实际可用能力填写
skill_set = []                 # 本示例未提供技能; 有技能时填写目录名
expected_tool_dependencies = ["filesystem", "shell", "python"] # 按实际必要能力填写
expected_skill_dependencies = [] # 必须是 skill_set 的子集
```

```

tags = ["finance", "office", "A2", "skill-dependency", "sop-
enforcement", "workflow", "docx", "llm-judge"]
# 逐题填写任务关键词

[[metadata.deliverables]]      # 交付物一
path = "FIN-T2-001_财务分析报告.xlsx"      # 交付物一的路径
required = true
desc = "主交付物：财务分析报告"      # 交付物一的描述

[[metadata.deliverables]]      # 见交付物一的说明，如有多个交付物，必须依次写明
path = "FIN-T2-001_毛利率趋势.png"
required = true
desc = "毛利率趋势折线图"

[agent]
timeout_sec = 72000.0      # 缺省 72000;

[verifier]
timeout_sec = 18000.0
user = "root"

[verifier.env]
# Judge 相关变量统一 JUDGE_ 前缀，只写在 [verifier.env]、严禁写入
[environment.env]。
# 必选项（运行环境注入，AP 必给）；可选项一律用 ${VAR:-默认值} 语法给默认值。
JUDGE_API_KEY = "${JUDGE_API_KEY:-}"      # 必选：judge 网关 key
JUDGE_BASE_URL = "${JUDGE_BASE_URL:-}"      # 必选：judge 网关地址（可
带 /v1, test.sh 负责剥）
JUDGE_MODEL = "${JUDGE_MODEL:-qwen3.7-plus}"      # 可选：裁判模型（经
REWARDKIT_MODEL 运行时生效）
JUDGE_PROVIDER = "${JUDGE_PROVIDER:-anthropic}" # 可选：信息性声明
JUDGE_API_PROTOCOL = "${JUDGE_API_PROTOCOL:-anthropic}" # 可选：openai
时降级为 LLM judge（见 test.sh）
# 兼容旧环境的兜底注入
EVAL_API_KEY = "${EVAL_API_KEY:-}"
EVAL_API_BASE = "${EVAL_API_BASE:-}"
LITELLM_DROP_PARAMS = "true"

[environment]
os = "linux"
build_timeout_sec = 18000.0
network_mode = "public"      # claude-code 框架下不要写 no-network（见
3.2 警示）
cpus = 2
memory_mb = 8192

```

```
storage_mb = 30720
```

照抄

TOML 书写： `**schema_version**`、`**artifacts**` 必须写在最前面；

先写 `**[metadata]**`，再写 `**[[metadata.deliverables]]**` / `**[[agent]]**` 等内容。

3.2 Task.toml 字段说明表

字段	说明	必填	约束
<code>[metadata].task_id</code>	题目编号	是	格式 领域缩写-任务类型-序号，如 FIN 1-001
<code>[metadata].domain</code>	所属领域（一级标签）	是	通用办公 / 金融 / 医疗 / 法律
<code>[metadata].domain_1_2</code>	所属领域二级标签	是	对齐知识体系表的领域二级标签
<code>[metadata].domain_1_3</code>	所属领域三级标签	是	对齐知识体系表的领域三级标签
<code>[metadata].domain_1_4</code>	所属领域四级标签	是	对齐知识体系表的领域四级标签
<code>[metadata].capabilities</code>	评测的能力	是	该任务能评测的具体专业能力
<code>[metadata].difficulty</code>	难度等级	是	A1（易）/ A2（中）/ A3（难）。按三模型平均正确率划分： A1 < 70%、 A2 < 60%、 A3 < 50%，验收标准见第 7 章
<code>[metadata].vl_dependency</code>	是否依赖 VL	是	是 / 否（VL 即视觉-语言能力）

<code>[metadata].source_note</code>	任务说明	是	为什么选这个任务、真实性如何保障、真实场景中的交付物是什么
<code>[metadata].tools</code>	所需工具	是	完成任务所需办公工具/数据库 (Word/Excel/PPT 等)
<code>[metadata].domain_knowledge</code>	领域知识依赖	否	完成任务所需的专业知识说明
<code>[metadata].environment_template</code>	环境模板名	本专项必填	string; 填写平台已确认的模板名, 与实际镜像依赖一致, 不替代 Dockerfile
<code>[metadata].tool_set</code>	模型可用的工具能力集合	本专项必填	array[string]; 使用平台已有工具名称, 如 <code>filesystem</code> 、 <code>shell</code> 、 <code>python</code> , 仅声明实际可用能力; 不要求三者分别对应独立原生工具
<code>[metadata].skill_set</code>	环境内可用技能	本专项必填	array[string]; 与 <code>environment/skills/<name>/</code> 逐一对应, 名称与 <code>SKILL.md</code> 中的 <code>name</code> 一致; 可含干扰技能, 无技能写 <code>[]</code>
<code>[metadata].expected_tool_dependencies</code>	真正必要的工具能力	本专项必填	array[string]; 必须是 <code>tool_set</code> 的子集, 无依赖写 <code>[]</code> ; 不是要求每种工具都调用一次

<code>[metadata].expected_skill_dependencies</code>	真正必要的技能	本专项必填	<code>array[string]</code> ；必须是 <code>skill_set</code> 的子集，可为空；含干扰技能的发现任务应为真子集，不能仅因安装了技能就认定存在依赖
<code>[metadata].task_complexity</code>	任务复杂度	本专项必填	<code>string</code> ； <code>C1</code> — <code>C5</code> ，按算法分级表填写，不从 <code>difficulty</code> 直接换算
<code>[metadata].weakness_tag</code>	覆盖的弱点标签	本专项必填	<code>array[string]</code> ；按算法材料的数组写法填写至少一项，取值使用正式词表
<code>[metadata].expected_pass_rate</code>	目标模型通过率预估	否	0~1
<code>[[metadata.deliverables]]</code>	参考答案（交付物清单）	是	见 3.3
<code>[environment].network_mode</code>	—	是	固定 <code>public</code> 。 claude-code 框架下不要写 <code>no-network</code> ：agent setup 阶段要联网安装 claude CLI，基线断网会直接 <code>AgentSetupTimeoutError</code> 、整题跑不起来。确需断网语义的题目，先与平台确认适配方案后再交
<code>[agent].timeout_seconds</code>	—	是	缺省 72000

以下格式照抄不改——Harbor 侧要求的字段，与验收无关，按下表要求机械填写，不要自行发挥。

字段	填法
<code>schema_version</code>	"1.4"
<code>artifacts</code>	由交付物清单机械展开：每个 <code>required = true</code> 的 <code>deliverables.path</code> 前面加 <code>/app/output/</code> ，不增不减、不改字
<code>[task].name</code>	work/fin1-001
<code>[task].version</code>	"1.0.0"
<code>[task].description</code>	一句话说明，可直接用 <code>instruction.md</code> 的标题
<code>[task].keywords</code>	[领域英文名, "office", 难度等级]，如 ["finance", "office", "A2"]
<code>[verifier].timeout_sec</code>	18000.0
<code>[verifier].user</code> / <code>[verifier].env</code>	逐字照 3.1 示例： <code>user = "root"</code> ； <code>judge</code> 相关变量统一 <code>JUDGE_</code> 前缀—— <code>JUDGE_API_KEY</code> / <code>JUDGE_BASE_URL</code> 必选（运行环境注入，不要替换成真实值）， <code>JUDGE_MODEL</code> 等可选项 必须用 <code>\${VAR:-默认值}</code> 语法给默认值。 <code>ANTHROPIC*/OPENAI</code> 不写进 <code>toml</code> ，由 <code>test.sh</code> 从 <code>JUDGE_</code> （兜底 <code>EVALAPI*</code> ）推导
<code>[environment].os</code> / <code>build_timeout_sec</code> / <code>cpus</code> / <code>memory_mb</code> / <code>storage_mb</code>	逐字照 3.1 示例；确需上调时在提交说明里注明原因

`[environment]` 只允许 Harbor 字段表内的键：`os` / `network_mode` / `allowed_hosts` / `build_timeout_sec` / `docker_image` / `cpus` / `memory_mb` / `storage_mb` / `env` / `healthcheck`。不要写 `**workdir**` ——工作目录由 `**environment/Dockerfile**` 的 `**WORKDIR /app*` 决定。

`[environment].env`（agent 侧环境变量）遵循**最小必要原则**：完成任务不需要的变量一律不配。`JUDGE_*` 只准写在 `[verifier].env`，严禁写入 `[environment].env` ——`judge` 凭据进 agent 环境等于把评分通道泄漏给被测模型。

3.3 [[metadata.deliverables]]

字段	类型	说明
<code>path</code>	string	相对 <code>/app/output/</code> 的文件名或 glob；分隔符用 <code>/</code>
<code>required</code>	bool	<code>true</code> = 缺失即判未完成交付； <code>false</code> = 可选产物
<code>desc</code>	string	一句话说明

命名与结构规则

#	规则	原因
1	<code>required = true</code> 必须写 精确文件名 ，禁止 glob / 通配 / "或等价命名"	glob 只允许用于 <code>required = false</code> 的过程性产物
2	文件名不得含 日期、时间戳、版本号 等动态成分	Agent 每次生成的名字不同，criteria 里写的交付物名永远对不上；业务上需含日期的，把具体字符串写死
3	大小写敏感 。清单、 <code>instruction.md</code> 、 <code>artifacts</code> 、 <code>solution/golden_output/</code> 、 <code>tests/_golden_output/</code> 、 criterion description 中引用的交付物路径 六处逐字节一致 （含全角/半角、空格、下划线/连字符）	Linux 容器内， <code>Report.docx</code> ≠ <code>report.docx</code>
4	需要目录层级时在 <code>path</code> 中写出； criterion description 里引用交付物时 逐个写完整文件路径 （如 <code>output/<文件名></code> ），不要只写目录	agent judge 靠 description 里的路径线索找文件，只给目录容易漏查文件、却照 criteria 判"没写"
5	UTF-8，单个文件名 ≤ 200 字节，无控制字符；建议以 题目编号 为前缀	避免与 Agent 中间产物混淆

3.4 Skill / Workflow 的使用说明

Skill 放在 `environment/skills/<技能名>/`，包含 `SKILL.md` 及所需脚本、资料。在已有 `[metadata]` 中声明，例如 AQ-001：

```
skill_set = ["contract-redliner"]
expected_skill_dependencies = ["contract-redliner"]
```



复制代码

```
skill_set = ["contract-redliner"]
expected_skill_dependencies = ["contract-redliner"]
```

Dockerfile 将技能复制到容器，`instruction.md` 列明技能入口（如 `/skills/contract-redliner/SKILL.md`）。Agent 读取说明后，通过已有工具执行脚本，不要求特定 harness 的加载方式。

Workflow 直接写在 `SKILL.md` 的 SOP 中，说明步骤、条件分支和完成检查，无需新增配置字段。例如 AQ 的“读取合同 → 修订 → 失败时重试失败项 → 最终复核”。

评分仍按原有交付物要求执行，不新增技能调用或流程顺序的过程评分。

4. environment/

4.1 单 Dockerfile 与参考模板

每题一个 `environment/Dockerfile`，公共依赖直接写全，全部走官方源（Debian / registry.npmjs.org / pypi.org）。禁止使用任何国内镜像源（npmmirror、aliyun pypi 镜像、docker registry mirror 等）：

- AP 平台部署在境外，官方源直连无加速需求，国内源反而更慢甚至不可达。

`environment/Dockerfile`（每题一个）：

```
FROM python:3.12-slim
```

```
RUN useradd -m -u 1000 agent
```

```
# ① node/npm 是 claude CLI 的运行依赖；中文字体 + libreoffice 供交付物渲染/校验
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
```

```
ca-certificates curl git bash jq ripgrep unzip nodejs npm \
```

```
libreoffice-calc libreoffice-writer fonts- noto-cjk \
```

```
&& rm -rf /var/lib/apt/lists/*
```

```
# ② 预装 claude-code（钉死版本 2.1.114）：judge 在 verifier 判分容器内运行，harbor
```

```

# 安装器不负责该容器，claude 必须镜像内预装。npm 走官方 registry（理由见 4.1 开头）。
# 版本必须钉死：评分行为随 CLI 版本漂移，不锁版本则跨批次分数不可比。
# 若基础镜像/上层环境已自带 claude，不覆盖——只在缺失时安装。
# ln 到 /root/.local/bin 的软链给 test.sh 的 PATH 前缀用（$HOME/.local/bin）。
RUN command -v claude >/dev/null 2>&1 \
    || npm install -g @anthropic-ai/claude-code@2.1.114; \
    command -v claude >/dev/null 2>&1 \
    && mkdir -p /root/.local/bin \
    && ln -sf "$(which claude)" /root/.local/bin/claude \
    && claude --version

# ③ pip 依赖：rewardkit + 基本的解析库（官方 pypi.org）
RUN pip install --no-cache-dir \
    "harbor-rewardkit[all]==0.1.7" markitdown \
    openpyxl pandas python-docx pypdf python-pptx PyYAML chardet matplotlib

# ④ 本题执行侧依赖（无依赖交空 requirements.txt）
COPY requirements.txt /tmp/requirements.txt
RUN pip install --no-cache-dir -r /tmp/requirements.txt

COPY input_files/ /app/input_files/
RUN chown -R root:root /app/input_files && chmod -R a-w /app/input_files

RUN mkdir -p /app/output && chown -R agent:agent /app/output

WORKDIR /app

```



复制代码

```
FROM python:3.12-slim
```

```
RUN useradd -m -u 1000 agent
```

① node/npm 是 claude CLI 的运行依赖；中文字体 + libreoffice 供交付物渲染/校验

```

RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates curl git bash jq ripgrep unzip nodejs npm \
    libreoffice-calc libreoffice-writer fonts-noto-cjk \
    && rm -rf /var/lib/apt/lists/*

```

② 预装 claude-code（钉死版本 2.1.114）：judge 在 verifier 判分容器内运行，harbor

```
# 安装器不负责该容器，claude 必须镜像内预装。npm 走官方 registry（理由见
4.1 开头）。
# 版本必须钉死：评分行为随 CLI 版本漂移，不锁版本则跨批次分数不可比。
# 若基础镜像/上层环境已自带 claude，不覆盖—只在缺失时安装。
# ln 到 /root/.local/bin 的软链给 test.sh 的 PATH 前缀用
($HOME/.local/bin)。
RUN command -v claude >/dev/null 2>&1 \
    || npm install -g @anthropic-ai/claude-code@2.1.114; \
    command -v claude >/dev/null 2>&1 \
    && mkdir -p /root/.local/bin \
    && ln -sf "$(which claude)" /root/.local/bin/claude \
    && claude --version

# ③ pip 依赖：rewardkit + 基本的解析库（官方 pypi.org）
RUN pip install --no-cache-dir \
    "harbor-rewardkit[all]==0.1.7" markitdown \
    openpyxl pandas python-docx pypdf python-pptx PyYAML chardet
matplotlib

# ④ 本题执行侧依赖（无依赖交空 requirements.txt）
COPY requirements.txt /tmp/requirements.txt
RUN pip install --no-cache-dir -r /tmp/requirements.txt

COPY input_files/ /app/input_files/
RUN chown -R root:root /app/input_files && chmod -R a-w /app/input_files

RUN mkdir -p /app/output && chown -R agent:agent /app/output

WORKDIR /app
```

若harbor题包依赖skills，镜像构建应增加以下内容：

▼

复制代码

```
COPY skills/ /skills/
```

skills所需依赖仍写入 `environment/requirements.txt`，按原模板在构建期安装。确保实际 Agent 用户能读取技能及引用资料、执行脚本，且任务书中的路径与镜像一致。

4.2 镜像硬性要求

要求	安装	自检
----	----	----

Python ≥ 3.12, 且有 <code>python3</code>	<code>FROM python:3.12-slim</code>	<code>python3 -V</code>
有 <code>bash</code>	基础镜像自带; <code>*-alpine</code> 需 <code>apk add --no-cache bash</code>	<code>bash --version</code>
有 <code>nodejs</code> / <code>npm</code> (<code>claude CLI</code> 运行依赖)	<code>apt-get install nodejs npm</code>	<code>node -v</code>
有 <code>claude</code> 可执行 (<code>judge</code> 用, <code>verifier</code> 容器内平台不代装)	见 4.1 模板 ② 预装块	<code>claude --version</code>
<code>harbor-rewardkit[all]==0.1.7</code> , 且 <code>rewardkit</code> 在 <code>root</code> 的 <code>PATH</code> 上	<code>pip install --no-cache-dir "harbor-rewardkit[all]==0.1.7"</code>	<code>rewardkit --version</code>
<code>pip</code> 已装文档解析库 (<code>markdown</code> / <code>openpyxl</code> / <code>python-docx</code> / <code>python-pptx</code> / <code>pypdf</code> 等) —— judge 读取交付物都依赖它们, 缺装会导致判官读不了二进制交付物、大面积判负	见 4.1 模板 ③④	<code>markdown --help</code> / <code>python3 -c "import openpyxl, docx, pptx, pypdf"</code>
存在 <code>agent</code> 用户, 且对 <code>/app/output</code> 可写	<code>useradd -m -u 1000 agent</code> + <code>chown -R agent:agent /app/output</code>	<code>su agent -c "touch /app/output/.w && rm /app/output/.w"</code>
依赖放入 <code>requirements.txt</code> 中	/	/

构建完成后整体自检:

```
docker build -t fin-t2-001 environment/
docker run --rm --network none fin-t2-001 bash -lc '
python3 -V && bash --version | head -1 && node -v &&
claude --version | grep -q 2.1.114 &&
rewardkit --help >/dev/null && markdown --help >/dev/null &&
python3 -c "import openpyxl, docx, pptx, pypdf" &&
pip show harbor-rewardkit | grep ^Version: && pip check &&
id agent && su agent -c "touch /app/output/.w && rm /app/output/.w" && echo OK'
```

[复制代码](#)

```
docker build -t fin-t2-001 environment/
docker run --rm --network none fin-t2-001 bash -lc '
python3 -V && bash --version | head -1 && node -v &&
claude --version | grep -q 2.1.114 &&
rewardkit --help >/dev/null && markitdown --help >/dev/null &&
python3 -c "import openpyxl, docx, pptx, pypdf" &&
pip show harbor-rewardkit | grep ^Version: && pip check &&
id agent && su agent -c "touch /app/output/.w && rm /app/output/.w"
&& echo OK'
```

末行打印 才算通过。

5. solution/

5.1 solve.sh

运行solve.sh时，应当执行如下逻辑：

```
#!/bin/bash
set -euo pipefail

mkdir -p /app/output
cp -R /solution/golden_output/. /app/output/
```

[复制代码](#)

```
#!/bin/bash
set -euo pipefail

mkdir -p /app/output
cp -R /solution/golden_output/. /app/output/
```

运行后，参考答案主分须 > 0.85 。

5.2 参考答案要求

要求

参考答案的文件名、格式、数量与 `instruction.md` 、 `[[metadata.deliverables]]` 内 严格一致

须满足 Rubric 的全部正分项
不得命中任何 <code>negate</code> 扣分条目
<code>solution/golden_output/</code> 与 <code>tests/__golden_output/</code> 内容一致，目录均不得为空

6. tests/（评分器）

6.1 运行链路（固定）

执行评分时，应当执行如下逻辑 → harness 执行 `bash /tests/test.sh` → 分数落到 `/logs/verifier/reward.json` 与 `/logs/verifier/reward.txt`（单一 int/float），逐条明细由 `finalize.py` 复制一份到 `/logs/verifier/reward-details.json`（审计用，缺失不影响主分）；评分不可用时按 AP 规范写 `/logs/verifier/reward_exit_message.json`（错误码枚举 `judge:timeout` / `judge:api_error` / `judge:parse_error` / `judge:scorer_error` / `judge:invalid_output` / `judge:unknown`，可多级）。

`test.sh` 与 `finalize.py` 是平台固定模板，逐字复制、含注释一并保留、不得改动。详见附录 A。

判官是 **claude-code agent judge**：以 CLI 进程跑在 verifier 容器内，自己用 Bash 工具读交付物（不再由 Reward Kit 预读文件塞进 prompt）。每条 **criterion** 单独起 1 个 **judge** 会话独立评分（`mode = "individual"`，逐条串行）。供应商负责 `rubrics.toml` 和 `prompt.md` 的内容。

6.2 rubrics.toml 与 prompt.md 示例

`tests/rubrics.toml`：

```
[judge]
judge = "claude-code" # 固定值，判官类型
prompt_template = "prompt.md" # 固定值，指向同目录 prompt.md
model = "qwen3.7-plus" # 裁判模型，照抄
timeout = 7200 # 单个 judge 会话（individual 模式下 = 每条 criterion 一个会话）的超时秒数
mode = "individual" # 每条评分点独立评分
weight = 1.0 # 固定值，照抄

[[criterion]]
id = "R1"
name = "R1"
description = "报告计算出近三年毛利率为 68% (±0.5pp)，并标注同比变动方向为正。"
```

Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."

type = "binary"

weight = 10.0

[[criterion]]

id = "R2"

name = "R2"

description = "估值表中企业价值 (EV) = 股权价值 + 净债务, 且净债务只计入一次。"

Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."

type = "binary"

weight = 7.0

[[criterion]]

id = "R3"

name = "R3"

description = "给出结论但缺少对应的数据支撑 (表格引用或可追溯的计算过程)。评分为 1-5 整数, 按违规程度判定 (negate 条目, Reward Kit 归一化后自动翻转计分): 5=绝大多数结论无任何数据支撑; 4=超过半数结论缺少数据支撑; 3=约半数结论缺少数据支撑; 2=仅个别结论缺少数据支撑; 1=所有结论均有表格引用或可追溯的计算过程。"

Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."

type = "likert"

points = 5

negate = true

weight = 7.0

[[criterion]]

id = "R4"

name = "R4"

description = "工作簿包含且仅包含五个工作表, sheet 名逐字为: 摘要、汇总表、毛利率、估值表、风险提示。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."

type = "binary"

weight = 3.0

[[criterion]]

id = "R5"

name = "R5"

description = "风险提示工作表覆盖四类风险 (客户集中度、应收账款账龄、汇率敞口、原材料价格), 且每条附量化依据。评分为 1-5 整数: 5=四类风险全部覆盖且每条均有可追溯的量化依据; 4=四类全覆盖, 但恰有 1 条缺量化依据; 3=覆盖 3 类风险且各有量化依据; 2=仅覆盖 1-2 类风险, 或多数条目缺量化依据; 1=风险提示工作表缺失, 或全部为无数据支撑的空泛描述。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."

type = "likert"


```
points = 5
weight = 7.0

[[criterion]]
id = "R7"
name = "R7"
description = "交付物存在致命问题：必交文件缺失、为空或完全不可用，或输出未脱敏的个人隐私/敏感数据（如身份证号、手机号）。仅当发现明确证据证明问题存在时判定成立。Deliverables to inspect: output/FIN-T2-001_财务分析报告.xlsx , output/FIN-T2-001_毛利率趋势.png ."
type = "binary"
negate = true
weight = 10.0

[scoring]
aggregation = "weighted_mean"
```

[复制代码](#)

```
[judge]
judge = "claude-code"           # 固定值，判官类型
prompt_template = "prompt.md"   # 固定值，指向同目录 prompt.md
model = "qwen3.7-plus"          # 裁判模型，照抄
timeout = 7200                  # 单个 judge 会话（individual 模式下 = 每条 criterion 一个会话）的超时秒数
mode = "individual"             # 每条评分点独立评分
weight = 1.0                    # 固定值，照抄

[[criterion]]
id = "R1"
name = "R1"
description = "报告计算出近三年毛利率为 68% (±0.5pp)，并标注同比变动方向为正。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."
type = "binary"
weight = 10.0

[[criterion]]
id = "R2"
name = "R2"
description = "估值表中企业价值（EV）= 股权价值 + 净债务，且净债务只计入一次。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."
type = "binary"
weight = 7.0

[[criterion]]
```

```

id = "R3"
name = "R3"
description = "给出结论但缺少对应的数据支撑（表格引用或可追溯的计算过程）。评分为 1-5 整数，按违规程度判定（negate 条目，Reward Kit 归一化后自动翻转计分）：5=绝大多数结论无任何数据支撑；4=超过半数结论缺少数据支撑；3=约半数结论缺少数据支撑；2=仅个别结论缺少数据支撑；1=所有结论均有表格引用或可追溯的计算过程。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."
type = "likert"
points = 5
negate = true
weight = 7.0

[[criterion]]
id = "R4"
name = "R4"
description = "工作簿包含且仅包含五个工作表，sheet 名逐字为：摘要、汇总表、毛利率、估值表、风险提示。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."
type = "binary"
weight = 3.0

[[criterion]]
id = "R5"
name = "R5"
description = "风险提示工作表覆盖四类风险（客户集中度、应收账款账龄、汇率敞口、原材料价格），且每条附量化依据。评分为 1-5 整数：5=四类风险全部覆盖且每条均有可追溯的量化依据；4=四类全覆盖，但恰有 1 条缺量化依据；3=覆盖 3 类风险且各有量化依据；2=仅覆盖 1-2 类风险，或多数条目缺量化依据；1=风险提示工作表缺失，或全部为无数据支撑的空泛描述。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`."
type = "likert"
points = 5
weight = 7.0

[[criterion]]
id = "R7"
name = "R7"
description = "交付物存在致命问题：必交文件缺失、为空或完全不可用，或输出未脱敏的个人隐私/敏感数据（如身份证号、手机号）。仅当发现明确证据证明问题存在时判定成立。Deliverables to inspect: `output/FIN-T2-001_财务分析报告.xlsx`，`output/FIN-T2-001_毛利率趋势.png`."
type = "binary"
negate = true
weight = 10.0

```

[scoring]

三条硬性写法（都对应真实故障或不可复现判分，缺一即整题判分崩溃、静默 0 分或档位随意漂移）：

- 每条 **criterion** 必须写 `name = "<id>"`（与 id 相同）。不写时 name 由 description 自动 slugify 生成，纯中文 description 会生成空串，全部条目键塌缩 → Reward Kit 解析崩溃。
- **description** 里必须写明待查交付物的完整路径（`Deliverables to inspect:` 清单，路径形如 `output/<文件名>`）。
- `type = "likert"` 的条目，必须显式写 `points = 5` 且在 description 里写明档位锚点——judge 被要求输出 **1 到 5 的整数**，锚点按 5/4/3/2/1 逐档写明对应的具体情形，见上例 R3/R5 与 6.6。不给锚点，judge 的中间档位给分不可复现、不可审计；锚点写成 0–1 小数则会被输出 schema (integer) 拒掉。

`tests/prompt.md`（可直接复制，把交付物格式按题目实际情况调整）：

You are an evaluation judge with filesystem access. Working directory: `/app`.
Evaluate the candidate's deliverables against the criteria at the end of this prompt.

[Material map]

`/app/output/` THE SUBJECT OF EVALUATION — the candidate's deliverables.

Only these files can earn or lose points.

`/app/input_files/` Task inputs given to the candidate (read-only). Consult to check whether deliverables are faithful to what was actually provided (e.g. a cited data source really exists; a stated fact is not fabricated).

`/tests/__golden_output/` One acceptable reference solution. See policy below.

[Reference-solution policy]

The reference is for calibration only — expected structure, field naming, magnitude of numbers. It is NOT an answer key and NOT a diff target. Two hard rules:

- Never award points because the reference satisfies a criterion. If the candidate's file lacks something, it lacks it.
- Never deduct for differing from the reference. Different wording, ordering, chart choices, or equally valid numbers are not wrong. Reference values are not ground truth unless the criterion says equality is required.

Where reference and criterion appear to disagree, the criterion wins.

[Tool usage — technical only, does NOT change scoring policy]

Inspect the deliverables however works best: shell commands, Python, any library in this container. Work out the approach per file type yourself; nothing here is a required route.

`markitdown <path>` is a handy one-step text extractor for `.xlsx/.docx/.pptx/.pdf`. This image was built for this task, so libraries needed for these deliverables are installed —

try importing before assuming one is missing. No network access; no Task/Explore subagents.

If a file genuinely cannot be opened by any available means, say so explicitly in your reasoning rather than silently treating it as missing or failing.

Fairness anchor:

None of the above changes how strictly you judge. Score each criterion exactly as the rubric prescribes; data extracted with any tool counts the same as reading the original. If a deliverable referenced by a criterion does not exist, judge per its description (typically false). Score only `/app/output/` — inputs and reference are evidence, never the thing being scored.

{criteria}



复制代码

You are an evaluation judge with filesystem access. Working directory: ``/app``. Evaluate the candidate's deliverables against the criteria at the end of this prompt.

[Material map].

<code>/app/output/</code>	THE SUBJECT OF EVALUATION – the candidate's deliverables.
<code>/app/input_files/</code>	Only these files can earn or lose points.
<code>/tests/__golden_output/</code>	Task inputs given to the candidate (read-only). Consult to check whether deliverables are faithful to what was actually provided (e.g. a cited data source really exists; a stated fact is not fabricated).
<code>/tests/__golden_output/</code>	One acceptable reference solution. See policy below.

[Reference-solution policy].

The reference is for calibration only – expected structure, field naming, magnitude of numbers. It is NOT an answer key and NOT a diff target. Two hard rules:

- Never award points because the reference satisfies a criterion. If the candidate's file lacks something, it lacks it.

- Never deduct for differing from the reference. Different wording, ordering, chart choices, or equally valid numbers are not wrong. Reference values are not ground truth unless the criterion says equality is required.

Where reference and criterion appear to disagree, the criterion wins.

[Tool usage – technical only, does NOT change scoring_policy].

Inspect the deliverables however works best: shell commands, Python, any library in this container. Work out the approach per file type yourself; nothing here is a required route.

`markitdown <path>` is a handy one-step text extractor for .xlsx/.docx/.pptx/.pdf. This image was built for this task, so libraries needed for these deliverables are installed – try importing before assuming one is missing. No network access; no Task/Explore subagents.

If a file genuinely cannot be opened by any available means, say so explicitly in your reasoning rather than silently treating it as missing or failing.

Fairness anchor:

None of the above changes how strictly you judge. Score each criterion exactly as the rubric prescribes; data extracted with any tool counts the same as reading the original. If a deliverable referenced by a criterion does not exist, judge per its description (typically false). Score only `/app/output/` – inputs and reference are evidence, never the thing being scored.

prompt.md 的六条规则：

1. ****{criteria}**** 占位符必须保留——Reward Kit 会把 criterion 列表渲染进去，删掉等于 judge 收不到评分项。
2. **Fairness anchor** 段不要删、也不要再在 prompt.md 里加任何评分宽严的表述——工具提示被误读成"放宽判分"会使判分严格度漂移。
3. **Material map** 段不要删——judge 的 cwd 是 `/app`，`input_files/` 与 `output/` 同级可见，`tests/` 被挂到 `/tests`（含 `golden_output/`）。三者都在 judge 视野内，模板沉默不等于禁止，只会把"要不要去翻参考答案"交给模型临场决定，同一批次内不同 criterion 会话的口径就此漂移。必须显式写明三区语义。

4. **Reference-solution policy** 段不要删、不得弱化——尤其"不得因参考答案满足就给分，一切以候选交付物为准"与"不得逐值比对"两条。前者防的是空产物得分（参考答案里有 ≠ 候选做到了）；后者防的是把参考解当唯一正确答案，对同样有效的不同解法误扣分。
5. **工具提示段**保持"提示"而非"强制"——列出容器内可用的解析途径（`markdown` CLI、`openpyxl` / `python-docx` / `python-pptx` / `pypdf` 等库）供 judge 选用，**不要写成"必须用某条命令"**：判官模型能力不一，部分模型可直接读 PDF 等格式，写死单一路径会挡掉更强的原生能力。环境确有的限制（如 Task / Explore 子代理不可用）照实告知即可，避免 judge 白耗 turn。
6. **总量有上限**：prompt.md 模板 + 全部 criterion description 之和须 < **100 KB**（prompt 经命令行参数传给 judge，超长会 E2BIG 直接起不来）。

`tests/__golden_output/` 是必交目录（见 1.2），在 verifier 容器内挂载为 `/tests/__golden_output/`，judge 可直接读取——因此 prompt.md 必须写明它的位置与使用政策（见上文规则 3、4 与 6.2 模板的 Reference-solution policy 段）：参考答案仅用于校准结构、字段命名、数值量级，**不得逐行/逐格/逐值比对打分，不得因参考答案满足就给分，一切以候选交付物为准**，rubric criterion 是得分的唯一权威。

6.3 criterion 字段约束

字段	约束
<code>[judge]. judge</code>	固定 <code>"claude-code"</code> 。
<code>[judge]. model</code>	裁判模型，合法 LiteLLM 模型串，按示例照抄（ <code>"qwen3.7-plus"</code> ）。运行时被 <code>REWARDKIT_MODEL</code> （ <code>test.sh</code> 由 <code>JUDGE_MODEL</code> 派生）覆盖，toml 值是无注入时的兜底
<code>[judge]. prompt_template</code>	固定 <code>"prompt.md"</code> ，指向同目录提示词文件（见 6.2）
<code>[judge]. timeout</code>	单个 judge 会话的超时秒数。缺省仅 300， 必须显式写 7200 ——agent judge 要反复调 Bash 读文件，远慢于一次性调用；会话超时不写 <code>reward.json</code> ，整题无分。 <code>[verifier]. timeout_sec</code> （18000）须大于本值
<code>[judge]. weight</code>	固定 <code>1.0</code> 照抄。不参与主分，只影响 Reward Kit 审计值
<code>[judge]. mode</code>	必写 <code>"individual"</code> ：每条 criterion 单独起一个 judge 会话评分（逐条串行）。

<code>[scoring].aggregation</code>	固定 <code>"weighted_mean"</code>
<code>[[criterion]].id</code>	必填，题内唯一，rubrics.json 中对应条目的编号保持一致，便于人工追溯
<code>[[criterion]].name</code>	必填，恒等于 id（缺失时中文 description slugify 成空串，解析崩溃，见 6.2）
<code>[[criterion]].description</code>	单条可独立判定的标准 + 待查交付物路径清单；likert 条目还须含各档位锚点。要求见 6.6
<code>[[criterion]].type</code>	只允许使用 <code>binary</code> 和 <code>likert</code> ；likert 时 judge 输出 1-5 整数（配合 <code>points = 5</code> ），必须在 description 按该标度写档位锚点（见 6.2 / 6.6）
<code>[[criterion]].points</code>	likert 必填，固定写 <code>5</code> （统一 5 档，锚点恒为 5/4/3/2/1）。不显式写时 Reward Kit 也按 5 处理，但必须写出，防止误设其它档数导致锚点与标度错位
<code>[[criterion]].weight</code>	只允许权重出现 <code>3.0</code> / <code>7.0</code> / <code>10.0</code> （见 6.4）。扣分只能用 <code>negate = true</code> + 正 weight，绝不能写负数 weight——负 weight 条目会被平台聚合脚本判为异常剔除并记评分不可用（ <code>verifier_error = 1</code> ），扣分静默失效、整次评分作废重评
<code>[[criterion]].negate</code>	<code>true</code> = 该条描述"候选犯了什么错"；判官判"存在"得 0、"不存在"得 1

6.4 权重档位

重要性	原分数	写法	落点
Critically Important	+10	<code>weight = 10.0 + type = "binary/likert"</code> ，	<code>rubrics.toml</code>
Important	+7	<code>weight = 7.0 + type = "binary/likert"</code> ，	<code>rubrics.toml</code>

Slightly Important	+3	<code>weight = 3.0 + type</code> <code>gate = "binary/likert"</code> ,	<code>rubrics.toml</code>
Slightly Detrimental	-3	<code>weight = 3.0 + negate</code> <code>gate = true + type</code> <code>= "binary/likert"</code> ,	<code>rubrics.toml</code>
Detrimental	-7	<code>weight = 7.0 + negate</code> <code>gate = true + type</code> <code>= "binary/likert"</code> ,	<code>rubrics.toml</code>
Critically Detrimental	-10	<code>weight = 10.0 + negate</code> <code>gate = true + type</code> <code>= "binary/likert"</code> ,	<code>rubrics.toml</code>

6.5 评分机制

由 `finalize.py` 全题池化，供应商不自行聚合。

$S_{\max}$ （满分基准）= $\sum$ 全部正向条目的 `weight` $\leftarrow$ 负向条目不进分母

分子 = $\sum$ 正向 `weight` $\times$ `value` - $\sum$ 负向 `weight` $\times$ (1 - `value`)

主分 `reward` = `clip`(分子 / $S_{\max}$, 0, 1)



复制代码

$S_{\max}$ （满分基准）= $\sum$ 全部正向条目的 `weight` $\leftarrow$ 负向条目不进分母

分子 = $\sum$ 正向 `weight` $\times$ `value` - $\sum$ 负向 `weight` $\times$ (1 - `value`)

主分 `reward` = `clip`(分子 / $S_{\max}$, 0, 1)

`S_max` 覆盖 `rubrics.toml`下的全部正向计分项。

`value` 是判官判定的归一化值：

type	value
<code>binary</code>	0 或 1

likert	judge 输出 1-5 的整数 （条目统一 <code>points = 5</code> ）；Reward Kit 归一化为 $(raw - 1) / 4$ ：1→0、2→0.25、3→0.5、4→0.75、5→1
<code>negate = true</code> 的条目	Reward Kit 自动翻转： 1 - 上述归一化值 （"完全没违规" = 1.0）

6.6 Rubric 细则要求

Rubric = 一个得分点/扣分点。模型回答到了我们希望的，得分；回答到了我们不希望的，扣分。

五项准则（每条 rubric 先过这五关）：

检查项	标准
原子性	每条判据仅检查一个方面（避免一条捆绑多个独立事实/多个可接受值）
客观性	判据为可客观验证的事实性陈述，不含"风格/美观"等过于主观的表述
区分度	正负样本明显区分；正分判据能区分"部分正确"与"完全正确"
完整性	覆盖多个维度
鲁棒性	多次 judge 判断基本稳定

- 打分项 = 一个得分点/扣分点，即考察交付物，
 - 符合某一条在实际生产中**被要求或有正向收益的打分项**，则得分；
 - 符合某一条实际生产中**应该杜绝或避免的打分项**，则扣分。
 - **注意：不能在同一个打分角度上，既写得分点，又写失分点，造成双重得分或双重扣分。**
- 每条打分项，**描述一个具体的、可验证的评分标准**，包含以下字段：

`description`：具体的评判标准描述。

- 单条打分项要求
 - **原子性**：每条打分项描述一个独立、明确、可验证的评分点，仅检查一个方面（避免一条捆绑多个独立事实/多个可接受值）。
 - **准确无歧义**：表述完整、事实正确、依据可靠，使不同评分者能够形成一致理解。

- **可独立判断**：打分项应包含作出判断所需的关键信息，不要求评分者额外推导、补充标准或进行二次判断。
 - 不推荐：收入实现了明显增长。
 - 推荐：收入同比增长了 65%。
- **来源合理**：打分项必须来自以下内容，不能凭空增加“伪需求”：
 - 项目描述中的明确要求；
 - 参考文件中的明确要求；
 - 根据任务目标和使用场景可以合理推导出的隐含要求。
- **有明确判定锚点**：禁止使用“语言流畅”“结构清晰”“内容专业”等空洞描述，应明确说明可观察、可验证的表现。
 - 不推荐：PPT 排版美观。
 - 推荐：单页正文不超过 150 字，文本无明显遮挡、重叠或溢出。
- **客观可验证**：优先评价能够直接核查的事实、数据、结构和结果，避免直接使用“美观”“深入”“专业”等主观词语。确需评价主观质量时，必须将其转化为具体的可观察标准。
- **粒度合理**：既不要把多个独立要求合并为一项，也不要将一个完整动作拆成大量无实际意义的小项。

• 打分项之间的要求

- **互不重复、互不冲突**：不同打分项应评价不同内容，避免同一问题被重复计分，也不能出现判定标准相互矛盾的情况。
 - 例如，“公文是否专业、有条理且合规”应拆分为：
 - 文种选择是否正确；
 - 格式要素是否齐全，如标题、主送机关、正文、落款、发文字号等；
 - 请求事项是否明确，是否符合“一文一事”。
- **具有区分度**：正分项和负分项的判定边界应明确；type为gradient项的，正分项要求能区分“部分满足”和“完全满足”，避免不同表现获得相同评价。
- **整体覆盖完整**：所有打分项合起来应覆盖任务的主要目标和关键能力，并能够从不同维度评价交付物质量，不能遗漏关键要求。
- **权重体现重要性**：同一份打分文件中，应根据各打分项对任务结果的影响程度设置不同的 `weight`，避免所有打分项机械地使用相同权重。
- **允许设置合理亮点项**：可以评价项目要求之外、但基于常识会影响用户判断或交付质量的内容。此类要求应具有明确价值和合理依据，不能无限扩展任务范围。
- **判断结果稳定（鲁棒性）**：同一套打分项由不同 Judge 或在不同时间多次评分时，结果应基本一致。若评分结果波动较大，应进一步明确描述、判定边界和示例。

critерion_type:

- Objective = 可度量、可验证，仅凭回答本身即可作出明确判定；
- Subjective = 需要打分员的判断与语境解读（语气/质量/风格，或对某理由、断言、示例是否成立的判断）。

- 字段约束： `criterion_type` $\in$ {Objective, Subjective}。

criterion_necessity:

- Explicit = 在 prompt 中逐字明文给出的要求；
- Implicit = 没有明说但从语境中必然推导出的要求。
- 字段约束： `criterion_necessity` $\in$ {Explicit, Implicit}。

权重weight

- 分值（正分表示奖励，负分表示扣分/惩罚）。
- 不接受恶意负分、故意增加题目难度/降低分数的行为。
- 字段约束为 `weight` $\in$ {+10、+7、+3、-3、-7、-10}，不能取任何其他值。
- 分值按下表取值：

级别	分数	含义
Critically Important	+10	不满足就是"答错大题"，核心要素
Important	+7	明显增色，但不一定是最低标准
Slightly Important	+3	细节/锦上添花
Slightly Detrimental	-3	小错误、小跑题
Detrimental	-7	重要错误，但整体还能用
Critically Detrimental	-10	致命错误，直接毁掉答案可信度

type

- 字段约束： `type` $\in$ {Binary, Gradient}。选择标准如下：

Binary（二元判定）	Gradient（分档位给分，接受中间状态）
答案要么满足、要么不满足；中间状态无意义	存在程度差异，且这种差异对最终质量有意义
主观但可明确 yes/no（如"是否有提及不确定性"，提及 or 不提及）	连续指标可以量化（数量、比例、偏离度）

- gradient（对应harbor的likert）打分项 示例

```
{
  "id": "R06",
```

```

    "description": "DCF 估值模型中使用的 WACC 取值应落在合理区间 [10%, 12%] 内，越接近得分越高",
    "dimension": "内容质量-数值与计算准确性",
    "criterion_type": "Objective",
    "criterion_necessity": "Explicit",
    "type": "gradient",
    "weight": 7.0,
    "levels": {
      "1": "WACC 落在 [10%, 12%] 区间内",
      "0.75": "WACC 偏离合理区间不超过  $\pm 1\%$ （如 9.0%–9.9% 或 12.1%–13.0%）",
      "0.5": "WACC 偏离合理区间  $\pm 1\%$ – $\pm 3\%$ （如 7.0%–8.9% 或 13.1%–15.0%）",
      "0.25": "WACC 偏离合理区间  $\pm 3\%$ – $\pm 5\%$ （如 5.0%–6.9% 或 15.1%–17.0%）",
      "0": "WACC 完全未给出，或偏离合理区间超过  $\pm 5\%$ "
    }
  }
}

```

[▾ gradient rubric示例](#)

[!\[\]\(2bdfe261b986065ee0ac76460d6528c9_img.jpg\) 复制代码](#)

```

{
  "id": "R06",
  "description": "DCF 估值模型中使用的 WACC 取值应落在合理区间 [10%, 12%] 内，越接近得分越高",
  "dimension": "内容质量-数值与计算准确性",
  "criterion_type": "Objective",
  "criterion_necessity": "Explicit",
  "type": "gradient",
  "weight": 7.0,
  "levels": {
    "1": "WACC 落在 [10%, 12%] 区间内",
    "0.75": "WACC 偏离合理区间不超过  $\pm 1\%$ （如 9.0%–9.9% 或 12.1%–13.0%）",
    "0.5": "WACC 偏离合理区间  $\pm 1\%$ – $\pm 3\%$ （如 7.0%–8.9% 或 13.1%–15.0%）",
    "0.25": "WACC 偏离合理区间  $\pm 3\%$ – $\pm 5\%$ （如 5.0%–6.9% 或 15.1%–17.0%）",
    "0": "WACC 完全未给出，或偏离合理区间超过  $\pm 5\%$ "
  }
}

```

• binary 打分项数据示例

```

{
  "id": "R17",
  "description": "计算2025年盈利净值为50万美元，相较于2024年盈利净值环比增长20%。",
  "dimension": "内容质量-数值与计算准确性",
  "criterion_type": "Objective",
  "criterion_necessity": "Explicit",

```

```
"type": "binary",
"weight": 3.0
}
```

binary rubric数据示例

复制代码

```
{
  "id": "R17",
  "description": "计算2025年盈利净值为50万美元，相较于2024年盈利净值环比增长20%。",
  "dimension": "内容质量-数值与计算准确性",
  "criterion_type": "Objective",
  "criterion_necessity": "Explicit",
  "type": "binary",
  "weight": 3.0
}
```

likert 条目必须显式写 `points = 5`，并在 `description` 里写明档位锚点。judge 对 likert 条目输出的是 **1 到 5 的整数**（Reward Kit 归一化为 $(\text{raw} - 1) / 4$ 后计入），锚点必须按 5/4/3/2/1 逐档写明可独立核验的具体情形，不留“judge 自由裁量”的空间。档数统一为 5，不接受其它 `points` 取值。方向约定：正向条目按满足程度写（5 = 完全满足 ... 1 = 完全不满足）；`negate = true` 的条目按**违规程度**写（5 = 违规完全成立，Reward Kit 归一化后翻转，计 0 分；1 = 完全无违规，翻转后满分）。示例（正向）：

```
[[criterion]]
id = "R6"
name = "R6"
description = "DCF 估值模型中使用的 WACC 取值应落在合理区间 [10%, 12%] 内，越接近得分越高。评分为 1-5 整数：5=WACC 落在 [10%, 12%] 区间内；4=偏离合理区间不超过 ±1%（如 9.0%-9.9% 或 12.1%-13.0%）；3=偏离 ±1%-±3%；2=偏离 ±3%-±5%；1=WACC 完全未给出，或偏离超过 ±5%。Deliverables to inspect: output/FIN-T2-001_财务分析报告.xlsx。"
type = "likert"
points = 5
weight = 7.0
```

复制代码

复制代码

```
[[criterion]]
id = "R6"
name = "R6"
description = "DCF 估值模型中使用的 WACC 取值应落在合理区间 [10%, 12%] 内，越接近得分越高。评分为 1-5 整数：5=WACC 落在 [10%, 12%] 区间内；4=偏离合理区间不超过 ±1%（如 9.0%-9.9% 或 12.1%-13.0%）；3=偏离 ±1%-±3%；2=偏离 ±3%-
```

```
±5%; 1=WACC 完全未给出，或偏离超过 ±5%。Deliverables to inspect:
`output/FIN-T2-001_财务分析报告.xlsx`."
type = "likert"
points = 5
weight = 7.0
```

数值算例（`points = 5`、`weight = 7.0`）：正向条 raw 5/4/3/2/1 → 计入 +7 / +5.25 / +3.5 / +1.75 / +0；`negate` 条按违规程度 raw 5/4/3/2/1 → 扣 7 / 5.25 / 3.5 / 1.75 / 0。

负向条目优先用 **binary**：违规通常"成立/不成立"没有中间态，且负向多档在现网无生产先例、判官一致性缺少验证；确需按违规程度分档扣分时才用 likert（如 6.2 的 R3），并把每一档写到可独立核验。

6.7 Rubric 的维度与数量要求

评价维度dimension

每条 打分项 需要有其所属评分维度。下面定义了综合考虑的维度，主要是引导大家设计打分项的时候思考全面，不遗漏重大维度，并且能有策略专注在价值维度上挖掘打分项。维度根据任务的要求来动态调整，不同的任务需要有区分度。

- 根据下表给出 打分项 评价的一级维度和二级维度，二级维度可为空。
- 不同维度的权重设置要求
 - 内容质量、操作与交付安全应该总是较高weight
 - 对于ppt等设计类交付物，视觉美感与格式规范维度应该较高weight
 - 对于行业性比较突出的交付物，专业规范维度应该较高weight
 - 对于写作类交付物，结构与组织维度应该较高weight
 - 对于数学计算要求较高的交付物，比如金融场景，数值与计算准确性维度应该较高weight

一级维度	二级标签	主要检查什么	典型条目
------	------	--------	------

指令遵循【必须】	检查 instruction 中显式要求的任务内容与交付约束是否被完整执行，包括范围限定、数量、命名、路径、格式、单位、精度以及明确要求包含或禁止的内容等。只检查 instruction 明确提出的要求；内容本身是否正确、专业、深入，归入对应内容质量维度	「结果文件名为 sales_summary.xlsx」「存放在 output/」「金额以万元为单位保留两位小数」 → 拆成 3 条
----------	---	--

内容质量【必须】	结论正确性	检查基于已有事实、数据和分析所形成的 最终判断、结论、推荐或决策是否正确 ，是否与证据一致，是否存在结论与数据相矛盾、判断方向错误或关键结论遗漏等问题。重点检查“最终得出了什么判断”，而非具体计算过程或论证深度。	数据显示 A 市场在增长率、利润率和竞争强度等关键指标上均优于 B 市场，最终推荐应为 A 市场，而不能错误推荐 B；
	数值与计算准确性	检查数值提取、公式计算、统计口径、单位换算、聚合方式、分母选择、时间范围、精度及容差等是否正确	合计行 = 各月之和 (± 0.01)；亿元/万元换算正确
	专业规范	检查内容是否符合任务所属行业 / 领域的专业知识、规则、标准和通行实践，包括会计准则、法律规则、医学规范、行业标准、专业术语及业务口径等	「应收账款」按照适用会计准则正确分类；法律分析使用现行有效的法律规则而非已失效规定；财务模型中的 EBITDA、FCF 等指标使用符合专业惯例的定义与口径
	分析与论证质量	检查产出中的事实、数据和引用是否忠实于输入材料及可验证的客观事实，不无依据地编造、篡改或混淆信息；引用和来源是否真实可追溯；跨文件处理时是否正确区分主体、版本、时间和统计口径。既检查 workspace 内事实，也检查任务涉及的外部客观事实。	「A 公司于 2016 年成立」这项产出表述能在 <code>industry_report.pdf</code> 表 3 溯源到确实有该项内容，或确实 A 公司于是 2016 年成立的，溯源不到即视为编造的幻觉内容，违反事实性。学术调研需求中，给出看似真实的论文标题、作者、期刊和 DOI，但实际查不到，为幻觉内容，需要检查。

	事实忠实性	检查产出中的事实、数据和引用是否忠实于输入材料及可验证的客观事实，不无依据地编造、篡改或混淆信息；引用和来源是否真实可追溯；跨文件处理时是否正确区分主体、版本、时间和统计口径。既检查 workspace 内事实，也检查任务涉及的外部客观事实。	「A 公司于 2016 年成立」这项产出表述能在 <code>industry_report.pdf</code> 表 3 溯源到确实有该项内容，或确实 A 公司是在 2016 年成立的，溯源不到即视为编造的幻觉内容，违反事实性。学术调研需求中，给出看似真实的论文标题、作者、期刊和 DOI，但实际查不到，为幻觉内容，需要检查。
	内容逻辑性与分析深度	分析到位、论证充分、有没有回答真问题、洞察深度。在指令遵循之外单独考察深度、逻辑、思想内涵。论点有准确论据/材料支撑。	销售额下滑分析给出 ≥ 2 条有数据支撑的归因，而不是复述「出现了下滑」
结构与组织		检查交付物的信息架构和组织方式是否合理，包括章节 / 页面结构是否完整、内容层级是否清晰、顺序是否符合逻辑、信息分组是否合理、关键模块是否缺失，以及文字、表格和图表是否放置在合理的上下文中	报告按「核心结论—分析依据—风险—建议」形成清晰结构
操作与交付安全		检查 Agent 在执行过程中是否安全处理用户已有文件和 workspace。不得误删、覆盖、破坏无关源文件，不得因编辑或格式转换造成数据、公式、页面或其他原有内容意外丢失	处理完成后 <code>data.csv</code> 仍完整可打开；被覆盖或删除则命中扣分项
安全合规		检查产出内容及执行行为是否满足隐私、数据安全、版权、法律法规及业务合规要求，避免泄露敏感信息、违规使用内容或产生明显误导、偏见及其他可能造成实际风险的问题。	对外报告不得直接暴露输入文件中的身份证号、手机号等敏感个人信息

超预期贡献		其他要求都满足之后，进一步提升交付价值的部分	事实二次核对、产物兼容性与易用性、防呆设计、指出 instruction 本身的疏漏以及更完善的交付物
视觉美感		交付物是否足够美观，比如排版、配色等内容是否让接手这份交付物的人感到欣赏，比较偏主观。（仅针对交付物有美观度要求的情况需要配置得分点，普通任务不建议包含）	PPT的配色和排版是否美观等

打分项分布要求

- 打分项的分值设计应符合统一标准，能够真实反映各能力的重要程度。主要包括：
 - **领域锚点占比**：交付物内容质量维度（交付物内容质量下所有二级标签）锚点正分，占全部正分的 **30%** 以上，表示本题对专业性有很高的要求，非简单指令遵循就可满分，要求有极强的领域知识理解。
 - **关键项占比**：每题至少包含 2 条 Critically Important (+10) 评分项，模拟用户真实期望，给出最影响产物本身可用性的打分点。
 - **-10分项**：仅用于重大专业错误、幻觉、业务安全、合规风险等真正影响答案可信度的情形，不得滥用。
- 一般来说，指令遵循、结论正确性、分析与论证质量与事实忠实性这些维度总是需要的。
 - 对于行业性比较突出的交付物，应该覆盖专业规范维度。
 - 对于ppt等设计类交付物，应该覆盖视觉美感与格式规范维度。
 - 对于写作类交付物，应该覆盖结构与组织维度。
 - 对于数学计算要求较高的交付物，比如金融场景，应该覆盖数值与计算准确性维度。

6.8 golden预检

题包提交后我方会进行 golden预检，不达标整题退回：

方向	做法	要求
正向	执行 <code>solve.sh</code> 后的 golden产物跑评分	主分 > 0.85 。不达标视为 Rubric 写歪，整题退回重写

本地自测命令：正向 `harbor run -p <题目目录> -a oracle` 。

7. 验收标准

• 单题计分公式：
$$\text{Score} = \frac{\sum_{i \in P} s_i \cdot w_i + \sum_{j \in N} h_j \cdot w_j}{\sum_{i \in P} w_i}$$

$w_i > 0$ ($i \in P$), P 正打分项; $w_j < 0$ ($j \in N$), N 负打分项

◦ $s_{\text{正}i} = \begin{cases} 1 & \text{binary 正打分项命中} \\ \{0, 0.25, 0.5, 0.75, 1\} & \text{gradient 正打分项实际档位} \end{cases}$

$h_{\text{负}j} = \begin{cases} 1 & \text{负向打分项命中 (产物出错)} \\ 0 & \text{负向打分项未命中 (无惩罚)} \end{cases}$

- **Claude code**框架下，跑 gpt-5.6-sol、claude-opus-4-8、qwen3.8-max0902三个模型，。所有题目满分归一化为 1.0 的情况下，对于GPT-5.6 sol / Opus 4.8 / Qwen3.8-max 0902 三个模型，裁判模型为qwen3.7 plus，每种模型都跑1次求平均分，均分需要 **< 0.7**；且至少有一个模型有得分（避免全 0 的"死题"）。
- 参考答案得分>0.85 & 三模型平均分 < 0.7。
- 不同等级的平均分符合该等级要求，A1介于0.6~0.7；A2介于0.5~0.6，A3<0.5
- 至少有一个模型得分。

8. 打包与提交

提交时，单题命名统一为：供应商名字+领域+一级分类+时间。如：xx-金融-投资银行-20260807提交。

多题一起提交时：压缩包命名为：供应商名字+领域+批次+时间。按照领域分别提交压缩包。

8.1 目录层级

固定为「批次目录 → 题目目录 → 五件套」，**不得多套一层**，也不得把题目目录平铺在 zip 根下。

```
供应商名字+领域+一级分类+时间/    ← 如xx-金融-投资银行-20260807提交。
├── 交付文档.md                      ← 环境变量配置说明等（见 8.4）
├── FIN-T2-001/
│   ├── instruction.md
│   ├── task.toml
│   ├── rubrics.json
│   ├── environment/
│   └── solution/
```

```

|   └── tests/
|   └── FIN-T2-002/
|   └── ...

```


[复制代码](#)

```

供应商名字+领域+一级分类+时间/
└── 交付文档.md
└── FIN-T2-001/
    ├── instruction.md
    ├── task.toml
    ├── rubrics.json
    ├── environment/
    ├── solution/
    └── tests/
└── FIN-T2-002/
└── ...

```

← 如xx-金融-投资银行-20260807提交。
 ← 环境变量配置说明等（见 8.4）

[task] 段缺失时可批量补齐，补齐后仍需**手工核对** name 是否符合 3.2 的命名规则（该命令按目录名生成 name）：

```

harbor task update "path/to/FIN-T2-001" --org "<供应商代号>"
harbor task update "path/to/tasks" --org "<供应商代号>" --scan # 整目录批量

```


[复制代码](#)

```

harbor task update "path/to/FIN-T2-001" --org "<供应商代号>"
harbor task update "path/to/tasks" --org "<供应商代号>" --scan # 整目录
批量

```

本地自测（`harbor run -p ...`）不需要登录 Harbor，供应商代号可自行拟定（建议用组织简拼，全批次固定，返修沿用）。

8.2 提交前自检

#	检查项
1	五件套齐全； <code>environment/requirements.txt</code> 即使无依赖也已交空文件
2	<code>solution/solve.sh</code> 、 <code>tests/test.sh</code> 为 LF 换行且带可执行位

3	交付物文件名在六处逐字节一致： <code>instruction.md</code> 、 <code>deliverables.path</code> 、 <code>artifacts</code> 、两份 <code>golden_output/</code> 、 <code>criterion description</code> 的交付物清单
4	满足 6.7 的条数下限、Critically Important ≥ 2 条、锚点 $\geq 30\%$ 、维度分布约束的"总是需要"维度已覆盖
5	<code>weight</code> 只出现 <code>3.0</code> / <code>7.0</code> / <code>10.0</code> 和 <code>20.0</code> ；无负数 <code>weight</code> ； <code>type</code> 只出现 <code>binary</code> / <code>likert</code> ； <code>likert</code> 条目均显式写 <code>points = 5</code> 且 <code>description</code> 含 5/4/3/2/1 档位锚点
6	本地跑通golden预检：Oracle > 0.85 ，且 <code>verifier_error = 0</code>
7	<code>task_id</code> 在目录名、 <code>[metadata].task_id</code> 、 <code>[task].name</code> 的 <code>name</code> 段三处一致（按小写连字符规则归一化后比较）； <code>[task].name</code> 的 <code>org</code> 段与批次目录前缀为同一代号
8	题包任何位置无真实密钥 / token / 凭证；需要时用占位符（如 <code><API_KEY></code> ）并在任务书说明。 <code>[environment].env</code> 中不得出现凭证或评分相关信息
9	无残留： <code>.git/</code> 、 <code>__pycache__/</code> 、 <code>.venv/</code> 、 <code>__MACOSX/</code> 、 <code>.DS_Store</code> ，以及本地跑测产生的 <code>reward.json</code> / <code>reward-details.json</code> / <code>logs/</code> / <code>jobs/</code>
10	所有文件名 UTF-8、单个 ≤ 200 字节、禁止符号链接；整包 ≤ 20 GB
11	<code>rubrics.toml</code> ： <code>judge = "claude-code"</code> ，每条 <code>criterion</code> 均有 <code>name</code> 且与 <code>id</code> 相同， <code>description</code> 带交付物路径清单；
12	<code>tests/prompt.md</code> 存在且含 <code>{criteria}</code> 占位符； <code>prompt.md</code> + 全部 <code>description</code> 总量 < 100 KB

13	镜像自检 (4.2) 未行打印 OK: <code>claude --version</code> 输出含 <code>2.1.114</code>
14	toml / prompt.md 中无不可见空白 (U+00A0 不换行空格、全角空格 U+3000 等——编辑器粘贴中文文案时易带入, 会导致 TOML 解析失败整题判分不可用)
15	<code>[verifier.env]</code> 中 <code>judge</code> 变量均为 <code>JUDGE_*</code> 前缀且可选项带 <code>\${VAR:-默认值}</code> ; <code>JUDGE_EXIT_CODE</code> 未出现在 <code>[environment].env</code> ; <code>[environment].env</code> 满足最小必要原则
16	本地跑完评分后 <code>/logs/verifier/reward.txt</code> 为单一数值且与 <code>reward.json</code> 的 <code>reward</code> 一致; <code>reward-details.json</code> 同时出现在 <code>/logs/verifier/</code> 与其 <code>graded/</code> 子目录且内容一致; 成功时不存在 <code>reward_exit_message.json</code> , 人为制造失败 (如清空 <code>JUDGE_API_KEY</code>) 时该文件出现且 <code>exit_code</code> 归类正确
17	批次根目录含交付文档 (8.4), 环境变量表覆盖全部实际使用的变量;

有 Skill / Workflow 的题目另检查: `skill_set` 与技能目录、任务书入口、Dockerfile 复制路径一致; 两个预期依赖集合满足子集关系; 以实际 Agent 用户验证技能可读、脚本可运行、SOP 的条件分支及复核说明完整。该检查属于环境与说明自检, 不增加过程评分, 也不替代原有 golden 预检。

8.3 提交与返修

整批打包成一个 **zip** 发给对接人, 随件注明批次目录名与题目数, 并附交付文档 (见 8.4, 放在批次根目录)。

返修重交: 修订题目的 `[task].version` 递增补丁号 (`1.0.0` → `1.0.1`), 只重交修订过的题目目录, 批次目录名沿用原名加 `_fix<N>` (`work_b01_20260805_fix1`), zip 名同步。供应商代号不随返修变更。

8.4 交付环境变量模板

批次根目录放一份交付文档 (`交付文档.md`), 包括但不限于环境变量配置说明——以表格逐个说明:

① Key 值；

② 是否必选，可选项写明默认值（数字型说明取值范围与单位；枚举型逐个说明枚举值含义）；

③ 与其他变量的联动关系。本规范标准题包的基线表如下，题包若有增补变量须一并列入：

Key	必选	默认值	类型/取值	说明与联动
JUDGE_API_KEY	是	—（运行环境注入）	string	judge 网关 key。 test.sh 派生为 ANTHROPIC_AUTH_TOKEN / ANTHROPIC_API_KEY / OPENAI_API_KEY
JUDGE_BASE_URL	是	—（运行环境注入）	string (URL)	judge 网关地址，可带 /v1 后缀 (test.sh 负责剥除后派生 ANTHROPIC_BASE_URL 等)
JUDGE_MODEL	否	qwen3.7-plus	string (LiteLLM 模型串)	裁判模型。经 REWARDKIT_MODEL 运行时覆盖 rubrics.toml 的 model；JUDGE_API_PROTOCOL=openai 时同时决定降级 LLM judge 的模型串
JUDGE_PROVIDER	否	anthropic	枚举：anthropic / openai	信息性声明，不改变行为

<code>JUDGE_API_PROTOCOL</code>	否	<code>anthropic</code>	枚举: <code>anthropic</code> / <code>openai</code>	<code>anthropic</code> = claude-code agent judge (默认链路); <code>openai</code> = 降级为 openai 协议 LLM judge (test.sh 设 <code>REWARDKIT_JUDGE=openai/<JUDGE_MODEL</code> > 覆盖 judge 类型, prompt.md 的工具提示随之不生效)
<code>EVAL_API_KEY</code> / <code>EVAL_API_BASE</code>	否	空	string	旧环境兼容兜底; <code>JUDGE*</code> 已注入时被忽略 (test.sh 里 <code>JUDGE*</code> 优先)
<code>LITELLM_DROP_PARAMS</code>	否	<code>true</code>	bool 字符串	网关兼容开关, 照抄

注意: judge 会话超时 (rubrics.toml `timeout = 7200`, 单位秒) 与 verifier 总超时 (task.toml `[verifier].timeout_sec = 18000`) 不经环境变量配置, rewardkit 无对应的运行时覆盖钩子, 调整须改 toml 并重新交付——此限制须在交付文档中如实说明。

附录 A：固定模板（逐字复制，含注释一并保留）

A.1 tests/test.sh

```
#!/bin/bash
# 平台固定模板，请勿改动，直接使用本模板。

set -uo pipefail

# --- judge 凭据: JUDGE_* (task.toml [verifier.env] 声明、平台注入) 优先,
# EVAL_API_* 为旧环境兜底 ---
_JUDGE_KEY="${JUDGE_API_KEY:-${EVAL_API_KEY:-}}"
_JUDGE_BASE="${JUDGE_BASE_URL:-${EVAL_API_BASE:-}}"
export ANTHROPIC_BASE_URL="${ANTHROPIC_BASE_URL:-$_JUDGE_BASE}"
export ANTHROPIC_BASE_URL="${ANTHROPIC_BASE_URL%/v1}"
export ANTHROPIC_AUTH_TOKEN="${ANTHROPIC_AUTH_TOKEN:-$_JUDGE_KEY}"
```



```

export ANTHROPIC_API_KEY="${ANTHROPIC_API_KEY:-${_JUDGE_KEY}}"
# --- OpenAI 兼容配置（若被降级回 LLM judge 时可用） ---
export OPENAI_API_KEY="${OPENAI_API_KEY:-${_JUDGE_KEY}}"
export OPENAI_BASE_URL="${OPENAI_BASE_URL:-${_JUDGE_BASE}}"
export OPENAI_API_BASE="${OPENAI_API_BASE:-${_JUDGE_BASE}}"
# --- 裁判模型运行时可配：rewardkit 原生识别 REWARDKIT_MODEL /
REWARDKIT_JUDGE ---
export REWARDKIT_MODEL="${REWARDKIT_MODEL:-${JUDGE_MODEL:-qwen3.7-
plus}}"
# JUDGE_API_PROTOCOL=openai 时降级为 openai 协议 LLM judge（覆盖 rubrics.toml
的 judge 类型）
if [ "${JUDGE_API_PROTOCOL:-anthropic}" = "openai" ]; then
    export REWARDKIT_JUDGE="${REWARDKIT_JUDGE:-
openai/${REWARDKIT_MODEL}}"
fi
export CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1
# claude-code 以 root 运行时拒绝 bypassPermissions（--dangerously-skip-permissions
# root 保护）；IS_SANDBOX=1 是官方逃生舱，容器化评测环境属预期场景。
export IS_SANDBOX=1
[ -f "$HOME/.local/bin/env" ] && source "$HOME/.local/bin/env"
export PATH="$HOME/.local/bin:/usr/local/bin:$PATH"

# rewardkit 调 claude 时不带 --permission-mode,
# 非交互模式默认权限下 Bash/Read 会被拒。
# 写 settings.json 强制 bypassPermissions（claude 启动时自动读取）。
mkdir -p "$HOME/.claude"
cat > "$HOME/.claude/settings.json" <<'SETTINGS'
{
  "permissions": {
    "defaultMode": "bypassPermissions"
  }
}
SETTINGS

if ! command -v rewardkit &> /dev/null; then
    echo "[test.sh] rewardkit executable not found; verifier image must preinstall harbor-
rewardkit" >&2
    exit 1
fi

mkdir -p /logs/verifier/graded

```

```
# fail-closed 兜底：判分器中途被杀（超时/OOM/容器回收）时，留下的必须是
# "评分不可用"，而不是一个有效 0 分——纯数字的 reward.txt 带不出"不可信"，
# 必须与 reward_exit_message.json 配合。finalize.py 正常收尾会覆盖这些文件
# （成功时删除错误文件）。
echo "0.0" > /logs/verifier/reward.txt
cat > /logs/verifier/reward.json <<'JSON'
{"graded_score": 0.0, "criteria_counted": 0.0,
 "reward": 0.0, "verifier_error": 1.0}
JSON
cat > /logs/verifier/reward_exit_message.json <<'JSON'
{"exit_code": "judge:unknown",
 "exit_reason": "verifier did not finish (killed before finalize.py); fail-closed placeholder",
 "extra_fields": {}}
JSON

# stderr 落盘供 finalize.py 归类错误码：rewardkit 除超时外的所有失败都是未捕获异常，
# 唯一的错误信息就是这里的 Python traceback（同时 tee 到 stdout 便于 test-stdout.txt
# 排查）。
rewardkit /tests --workspace /app --output /logs/verifier/graded/reward.json \
  2> >(tee /logs/verifier/graded/stderr.txt >&2)
graded_rc=$?

python3 /tests/finalize.py \
  --graded /logs/verifier/graded/reward.json --graded-rc "$graded_rc" \
  --out /logs/verifier/reward.json
```



复制代码

```
#!/bin/bash
# 平台固定模板，请勿改动，直接使用本模板。

set -uo pipefail

# --- judge 凭据：JUDGE_*（task.toml [verifier.env] 声明、平台注入）优先，
# EVAL_API_* 为旧环境兜底 ---
_JUDGE_KEY="${JUDGE_API_KEY:-${EVAL_API_KEY:-}}"
_JUDGE_BASE="${JUDGE_BASE_URL:-${EVAL_API_BASE:-}}"
export ANTHROPIC_BASE_URL="${ANTHROPIC_BASE_URL:-$_JUDGE_BASE}"
export ANTHROPIC_BASE_URL="${ANTHROPIC_BASE_URL%/v1}"
export ANTHROPIC_AUTH_TOKEN="${ANTHROPIC_AUTH_TOKEN:-$_JUDGE_KEY}"
export ANTHROPIC_API_KEY="${ANTHROPIC_API_KEY:-$_JUDGE_KEY}"
# --- OpenAI 兼容配置（若被降级回 LLM judge 时可用） ---
export OPENAI_API_KEY="${OPENAI_API_KEY:-$_JUDGE_KEY}"
export OPENAI_BASE_URL="${OPENAI_BASE_URL:-$_JUDGE_BASE}"
```

```

export OPENAI_API_BASE="${OPENAI_API_BASE:-${_JUDGE_BASE}}"
# --- 裁判模型运行时可配: rewardkit 原生识别 REWARDKIT_MODEL /
REWARDKIT_JUDGE ---
export REWARDKIT_MODEL="${REWARDKIT_MODEL:-${JUDGE_MODEL:-qwen3.7-
plus}}"
# JUDGE_API_PROTOCOL=openai 时降级为 openai 协议 LLM judge (覆盖
rubrics.toml 的 judge 类型)
if [ "${JUDGE_API_PROTOCOL:-anthropic}" = "openai" ]; then
    export REWARDKIT_JUDGE="${REWARDKIT_JUDGE:-openai/${REWARDKIT_MODEL}}"
fi
export CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1
# claude-code 以 root 运行时拒绝 bypassPermissions (--dangerously-skip-
permissions
# root 保护); IS_SANDBOX=1 是官方逃生舱, 容器化评测环境属预期场景。
export IS_SANDBOX=1
[ -f "$HOME/.local/bin/env" ] && source "$HOME/.local/bin/env"
export PATH="$HOME/.local/bin:/usr/local/bin:$PATH"

# rewardkit 调 claude 时不带 --permission-mode,
# 非交互模式默认权限下 Bash/Read 会被拒。
# 写 settings.json 强制 bypassPermissions (claude 启动时自动读取)。
mkdir -p "$HOME/.claude"
cat > "$HOME/.claude/settings.json" <<'SETTINGS'
{
    "permissions": {
        "defaultMode": "bypassPermissions"
    }
}
SETTINGS

if ! command -v rewardkit &> /dev/null; then
    echo "[test.sh] rewardkit executable not found; verifier image must
preinstall harbor-rewardkit" >&2
    exit 1
fi

mkdir -p /logs/verifier/graded

# fail-closed 兜底: 判分器中途被杀 (超时/OOM/容器回收) 时, 留下的必须是
# "评分不可用", 而不是一个有效 0 分—纯数字的 reward.txt 带不出"不可信",
# 必须与 reward_exit_message.json 配合。finalize.py 正常收尾会覆盖这些文件
# (成功时删除错误文件)。
echo "0.0" > /logs/verifier/reward.txt
cat > /logs/verifier/reward.json <<'JSON'
{"graded_score": 0.0, "criteria_counted": 0.0,

```

```

    "reward": 0.0, "verifier_error": 1.0}
JSON
cat > /logs/verifier/reward_exit_message.json <<'JSON'
{"exit_code": "judge:unknown",
 "exit_reason": "verifier did not finish (killed before finalize.py);
fail-closed placeholder",
 "extra_fields": {}}
JSON

# stderr 落盘供 finalize.py 归类错误码: rewardkit 除超时外的所有失败都是未捕获异常,
# 唯一的错误信息就是这里的 Python traceback (同时 tee 到 stdout 便于 test-stdout.txt 排查)。
rewardkit /tests --workspace /app --output
/logs/verifier/graded/reward.json \
    2> >(tee /logs/verifier/graded/stderr.txt >&2)
graded_rc=$?

python3 /tests/finalize.py \
    --graded /logs/verifier/graded/reward.json --graded-rc "$graded_rc" \
    --out /logs/verifier/reward.json

```

A.2 tests/finalize.py

```

#!/usr/bin/env python3
"""平台固定模板，请勿改动。

```

从 Reward Kit 的逐条判定明细汇总主分（按签名权重池化全题 criterion），并显式区分"评分不可用"与"确实得零分"。

```

"""

```

```

import argparse
import sys
import json
import math
import pathlib
import re

```

```

def load_json(path):
    """读取 JSON；不可读或解析失败返回 None。"""
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except Exception:
        return None

```

```
def load_scores(path):
    data = load_json(path)
    return data if isinstance(data, dict) else None
```

```
def finite(value):
    """转成有限浮点数；不可转、NaN、±inf 一律返回 None。
```

Reward Kit 只对 judge criterion 归一化到 [0, 1]，程序化 criterion 的返回值不钳制（越界只 warn），NaN / inf 会原样写进明细。这类值若直接参与运算会算出一个 0 分，看起来像"确实得零分"，必须当成评分异常上报。

```
"""
try:
    number = float(value)
except (TypeError, ValueError):
    return None
return number if math.isfinite(number) else None
```

```
def iter_criteria(details):
    """遍历明细里的全部 criterion。
```

details[<维度>] 在该维度只有一个 Reward 时是 dict；judge TOML 与 .py 混用、或放了多份 judge TOML 时是 list。两种形状都要处理。

```
"""
if not isinstance(details, dict):
    return
for entry in details.values():
    blocks = entry if isinstance(entry, list) else [entry]
    for block in blocks:
        if not isinstance(block, dict):
            continue
        for item in block.get("criteria") or []:
            if isinstance(item, dict):
                yield item
```

```
def pooled_score(details):
    """全题池化的签名加权分，返回 (分数, 参与条数, 异常条数)。
```

正向项: +weight 进分子、weight 进分母。

negate 项: -weight 进分子、不进分母。明细里的 value 是翻转后的值

(违规存在 = 0) , 违规程度需还原为 1 - value。

异常条目一律不计入、改由 verifier_error 上报, 包括: 带 error (判官超时会把每条都记成 value = 0.0 并保留 negate, 若计入会凭空扣分)、weight 非正数、value 非有限值、negate 非布尔值。

无正向条目时分母为 0, 主分无定义, 返回 (None, ...)。

```
"""
```

```
numerator = 0.0
```

```
denominator = 0.0
```

```
counted = 0
```

```
broken = 0
```

```
for item in iter_criteria(details):
```

```
    weight = finite(item.get("weight"))
```

```
    value = finite(item.get("value"))
```

```
    negate = item.get("negate")
```

```
    if (item.get("error") or weight is None or weight <= 0.0
```

```
        or value is None or not isinstance(negate, (bool, type(None)))):
```

```
        broken += 1
```

```
        continue
```

```
    value = min(1.0, max(0.0, value)) # 程序化 criterion 越界返回值的兜底
```

```
    if negate:
```

```
        numerator -= weight * (1.0 - value)
```

```
    else:
```

```
        numerator += weight * value
```

```
        denominator += weight
```

```
    counted += 1
```

```
if denominator <= 0.0:
```

```
    return None, counted, broken
```

```
return min(1.0, max(0.0, numerator / denominator)), counted, broken
```

```
def count_errors(node):
```

```
    """递归统计明细里的 error 字段。judge 超时 / 限额会被记成 0.0 加 error。"""
```

```
    total = 0
```

```
    if isinstance(node, dict):
```

```
        for key, value in node.items():
```

```
            if key == "error" and value:
```

```
                total += 1
```

```
            else:
```

```
                total += count_errors(value)
```

```
    elif isinstance(node, list):
```

```

    for item in node:
        total += count_errors(item)
    return total

```

```

def detail_errors(reward_path):
    """扫描 reward.json 同目录下的 *details*.json。"""
    total = 0
    for path in sorted(reward_path.parent.glob("*details*.json")):
        data = load_json(path)
        if data is None:
            total += 1
        else:
            total += count_errors(data)
    return total

```

```

def detail_error_messages(reward_path):
    """收集明细里全部 error 字符串。

```

Reward Kit 只在**判官超时**这一种情况下写 error 字段（judges.py 的 ``f"judge timed out after {timeout}s"``），其余失败都是未捕获异常，错误信息只在 stderr 的 traceback 里。

```

"""

```

```

msgs = []

```

```

def walk(node):
    if isinstance(node, dict):
        for key, value in node.items():
            if key == "error" and value:
                msgs.append(str(value))
            else:
                walk(value)
    elif isinstance(node, list):
        for item in node:
            walk(item)

```

```

for path in sorted(reward_path.parent.glob("*details*.json")):
    walk(load_json(path))
return msgs

```

```
def read_stderr_tail(reward_path, limit=2000):
    """读 test.sh 落盘的 rewardkit stderr 末尾（Python traceback 的末行信息量最大）。"""
    try:
        text = (reward_path.parent / "stderr.txt").read_text(
            encoding="utf-8", errors="replace").strip()
    except Exception:
        return ""
    return text[-limit:]
```

Python traceback 的**最后一条异常行**才是真正的错误信息（前面全是栈帧）。
 # rewardkit 用 ExceptionGroup 包裹异常，每行带 " | " 前缀，一并剥掉。
 _ERROR_LINE_RE = re.compile(
 r"^\[s|\+]*((?:\w+\.)*\w*(?:Error|Exception|Timeout)\b.*)\$", re.MULTILINE)

```
def last_error_line(text):
    """从 traceback 里取最后一条异常行；取不到返回空串。"""
    matches = _ERROR_LINE_RE.findall(text or "")
    return matches[-1].strip() if matches else ""
```

Reward Kit 抛出的异常类型 → AP 错误码。左侧字符串取自 rewardkit 0.1.7 源码里
 # 逐字写死的异常消息，不是猜测；未命中的一律按 scorer_error 兜底并透传原文。

```
_EXIT_CODE_RULES = (
    ("timed out after", "judge:timeout"),          # judges.py 超时（error 字
    段/warning)
    ("Could not parse JSON from judge response", "judge:parse_error"),
    ("expected dict with 'score' and 'reasoning'", "judge:parse_error"),
    ("exited with code", "judge:scorer_error"),      # agent CLI 非零退出
    ("RateLimitError", "judge:api_error:rate_limit"), # litellm 异常类名
    ("ContentPolicyViolationError", "judge:api_error:content_filter"),
    ("AuthenticationError", "judge:api_error:auth"),
    ("litellm", "judge:api_error"),
)
```

```
def classify_exit(args, result):
    """归类为 AP 错误码，并把 Reward Kit 的原始错误文本原样透传进 exit_reason。
```

分类只做三件确定的事：命中源码里写死的异常消息 → 对应码；rewardkit 非零退出

→ scorer_error; 跑通了却没有任何有效条目 → invalid_output。其余 unknown。无论哪种, exit_reason 都是 Reward Kit 自己的原话 (error 字段或 stderr 末尾), 不做二次加工——排查时看到的应当是判分器实际报了什么。

```
"""
```

```
detail_msgs = detail_error_messages(pathlib.Path(args.graded))
stderr_tail = read_stderr_tail(pathlib.Path(args.graded))
haystack = " | ".join(detail_msgs) + "\n" + stderr_tail
reason = (detail_msgs[0] if detail_msgs else
          last_error_line(stderr_tail) or stderr_tail[-1000:]) or (
    "verifier marked unavailable without any error output")
for needle, code in _EXIT_CODE_RULES:
    if needle.lower() in haystack.lower():
        return code, reason
if args.graded_rc != 0:
    return "judge:scorer_error", reason
if not result.get("criteria_counted"):
    return "judge:invalid_output", reason
return "judge:unknown", reason
```

```
def compute(args):
```

```
    """汇总评分结果, 返回要写进 reward.json 的字典。"""
```

```
    graded_path = pathlib.Path(args.graded)
```

```
    graded = load_scores(graded_path)
```

```
    graded_details = load_json(graded_path.with_name("reward-details.json"))
```

```
    dims = {}
```

```
    for key, value in (graded or {}).items():
```

```
        if key == "soft_score":
```

```
            continue
```

```
            number = finite(value)
```

```
            if number is not None:
```

```
                dims[key] = number
```

```
    # 主分: 按签名权重池化全题 criterion (负向项真扣分, 空产物下限为 0)。
```

```
    pooled, counted, broken = pooled_score(graded_details)
```

```
    score = 0.0 if pooled is None else round(pooled, 6)
```

```
    # Reward Kit 自己的 [0,1] 归一化聚合值, 仅留作审计参照, 不作主分。
```

```
    soft = finite((graded or {}).get("soft_score"))
```

```
    if soft is not None:
```

```

soft = round(soft, 6)

graded_ok = (args.graded_rc == 0 and bool(dims) and pooled is not None
             and counted > 0 and broken == 0)

result = dict(dims)
result["graded_score"] = score
result["criteria_counted"] = float(counted)
if soft is not None:
    result["soft_score"] = soft
# 评分不可用时主分一律记 0：宁可保守低估，也不要因为把异常条目排除在分母之外
# 而把剩下的条目重新归一化成一个虚高的分数。真实分数留在 graded_score 里。
unavailable = not graded_ok
result["reward"] = 0.0 if unavailable else score
# 平台读取：1 = 本次评分不可信（判官限额/超时/评分器异常），须重评而非记零分。
result["verifier_error"] = 1.0 if unavailable else 0.0
return result

def main() -> int:
    # 被 rewardkit discover() import 时不得有副作用：它会把 tests/ 下所有 *.py
    # 都 import 一遍。若 argparse 留在模块级，import 即 SystemExit 杀死评分进程。
    parser = argparse.ArgumentParser()
    parser.add_argument("--graded", required=True)
    parser.add_argument("--graded-rc", type=int, required=True)
    parser.add_argument("--out", required=True)
    args = parser.parse_args()

    try:
        result = compute(args)
    except Exception:
        # 未预期的异常也必须落地一份结果：缺了 reward.json，平台读到的是"这道题没跑
        # 过"，
        # 与"跑出 0 分"无法区分。一律记 verifier_error = 1 交平台重评。
        result = {"graded_score": 0.0, "criteria_counted": 0.0,
                  "reward": 0.0, "verifier_error": 1.0}

    out_path = pathlib.Path(args.out)
    out_path.parent.mkdir(parents=True, exist_ok=True)
    out_path.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-
8")

```

```

# ---- AP 标准化输出 ----
# reward.txt: 单一数值（覆盖 test.sh 开头的 fail-closed 占位）。
out_path.with_name("reward.txt").write_text(
    f"{result.get('reward', 0.0)}\n", encoding="utf-8")
# reward_exit_message.json: 仅评分不可用时存在；成功则删除 fail-closed 占位。
exit_path = out_path.with_name("reward_exit_message.json")
if result.get("verifier_error"):
    try:
        code, reason = classify_exit(args, result)
    except Exception:
        code, reason = "judge:unknown", "classification itself failed"
    exit_path.write_text(json.dumps({
        "exit_code": code,
        "exit_reason": reason,
        "extra_fields": {
            "criteria_counted": result.get("criteria_counted", 0.0),
            "graded_score": result.get("graded_score", 0.0),
            "rewardkit_rc": args.graded_rc,
        },
    }, ensure_ascii=False, indent=2), encoding="utf-8")
else:
    exit_path.unlink(missing_ok=True)
# reward-details.json: 逐条明细随主分一并落到 out 同目录（rewardkit 写在
# --output 旁的 graded/ 子目录）。审计便利件，不参与 fail-closed 契约：
# 缺失或拷贝失败不影响主分与错误码归类，静默跳过。
graded_details = pathlib.Path(args.graded).with_name("reward-details.json")
if graded_details.is_file():
    try:
        out_path.with_name("reward-details.json").write_text(
            graded_details.read_text(encoding="utf-8"), encoding="utf-8")
    except OSError:
        pass
# 退出码恒 0：评分不可用由 reward.json 的 verifier_error 承载，
# 非零退出会被平台当成 finalize 自身崩溃、丢掉已写好的结果。
return 0

if __name__ == "__main__":
    sys.exit(main())

```



复制代码

```

#!/usr/bin/env python3
"""平台固定模板，请勿改动。

```

从 **Reward Kit** 的逐条判定明细汇总主分（按签名权重池化全题 **criterion**），并显式区分"评分不可用"与"确实得零分"。

```
"""
import argparse
import sys
import json
import math
import pathlib
import re

def load_json(path):
    """读取 JSON；不可读或解析失败返回 None。"""
    try:
        return json.loads(path.read_text(encoding="utf-8"))
    except Exception:
        return None

def load_scores(path):
    data = load_json(path)
    return data if isinstance(data, dict) else None

def finite(value):
    """转成有限浮点数；不可转、NaN、±inf 一律返回 None。

    Reward Kit 只对 judge criterion 归一化到 [0, 1]，程序化 criterion 的返回值
    钳制（越界只 warn），NaN / inf 会原样写进明细。这类值若直接参与运算会算
    出一个 0 分，看起来像"确实得零分"，必须当成评分异常上报。
    """
    try:
        number = float(value)
    except (TypeError, ValueError):
        return None
    return number if math.isfinite(number) else None

def iter_criteria(details):
    """遍历明细里的全部 criterion。
```

`details[<维度>]` 在该维度只有一个 `Reward` 时是 `dict`; `judge TOML` 与 `.py` 混用、或放了多份 `judge TOML` 时是 `list`。两种形状都要处理。

```

"""
if not isinstance(details, dict):
    return
for entry in details.values():
    blocks = entry if isinstance(entry, list) else [entry]
    for block in blocks:
        if not isinstance(block, dict):
            continue
        for item in block.get("criteria") or []:
            if isinstance(item, dict):
                yield item

def pooled_score(details):
    """全题池化的签名加权分，返回（分数，参与条数，异常条数）。

    正向项: +weight 进分子、weight 进分母。
    negate 项: -weight 进分子、不进分母。明细里的 value 是翻转后的值
        （违规存在 = 0），违规程度需还原为 1 - value。
    异常条目一律不计入、改由 verifier_error 上报，包括：带 error（判官超时会把
    每条都记成 value = 0.0 并保留 negate，若计入会凭空扣分）、weight 非正
    数、
    value 非有限值、negate 非布尔值。
    无正向条目时分母为 0，主分无定义，返回 (None, ...)。
    """
    numerator = 0.0
    denominator = 0.0
    counted = 0
    broken = 0
    for item in iter_criteria(details):
        weight = finite(item.get("weight"))
        value = finite(item.get("value"))
        negate = item.get("negate")
        if (item.get("error") or weight is None or weight <= 0.0
            or value is None or not isinstance(negate, (bool,
type(None)))):
            broken += 1
            continue
        value = min(1.0, max(0.0, value)) # 程序化 criterion 越界返回值的兜底
        if negate:

```

```

        numerator -= weight * (1.0 - value)
    else:
        numerator += weight * value
        denominator += weight
    counted += 1
if denominator <= 0.0:
    return None, counted, broken
return min(1.0, max(0.0, numerator / denominator)), counted, broken

```

```

def count_errors(node):
    """递归统计明细里的 error 字段。judge 超时 / 限额会被记成 0.0 加 error。"""

```

```

    total = 0
    if isinstance(node, dict):
        for key, value in node.items():
            if key == "error" and value:
                total += 1
            else:
                total += count_errors(value)
    elif isinstance(node, list):
        for item in node:
            total += count_errors(item)
    return total

```

```

def detail_errors(reward_path):
    """扫描 reward.json 同目录下的 *details*.json。"""
    total = 0
    for path in sorted(reward_path.parent.glob("*details*.json")):
        data = load_json(path)
        if data is None:
            total += 1
        else:
            total += count_errors(data)
    return total

```

```

def detail_error_messages(reward_path):
    """收集明细里全部 error 字符串。

```

Reward Kit 只在**判官超时**这一种情况下写 error 字段（judges.py 的 ``f"judge timed out after {timeout}s"``），其余失败都是未捕获异常，错误信息只在 stderr 的 traceback 里。

```

"""

```

```

msgs = []

def walk(node):
    if isinstance(node, dict):
        for key, value in node.items():
            if key == "error" and value:
                msgs.append(str(value))
            else:
                walk(value)
    elif isinstance(node, list):
        for item in node:
            walk(item)

for path in sorted(reward_path.parent.glob("*details*.json")):
    walk(load_json(path))
return msgs

def read_stderr_tail(reward_path, limit=2000):
    """读 test.sh 落盘的 rewardkit stderr 末尾（Python traceback 的末行信息量最大）。"""
    try:
        text = (reward_path.parent / "stderr.txt").read_text(
            encoding="utf-8", errors="replace").strip()
    except Exception:
        return ""
    return text[-limit:]

# Python traceback 的**最后一条异常行**才是真正的错误信息（前面全是栈帧）。
# rewardkit 用 ExceptionGroup 包裹异常，每行带 " | " 前缀，一并剥掉。
_ERROR_LINE_RE = re.compile(
    r"^\[s|+]*((?:\w+\.)*\w*(?:Error|Exception|Timeout)\b.*)$",
    re.MULTILINE)

def last_error_line(text):
    """从 traceback 里取最后一条异常行；取不到返回空串。"""
    matches = _ERROR_LINE_RE.findall(text or "")
    return matches[-1].strip() if matches else ""

# Reward Kit 抛出的异常类型 → AP 错误码。左侧字符串取自 rewardkit 0.1.7 源码里
# 逐字写死的异常消息，不是猜测；未命中的一律按 scorer_error 兜底并透传原文。

```

```

_EXIT_CODE_RULES = (
    ("timed out after", "judge:timeout"),          #
judges.py 超时 (error 字段/warning)
    ("Could not parse JSON from judge response", "judge:parse_error"),
    ("expected dict with 'score' and 'reasoning'", "judge:parse_error"),
    ("exited with code", "judge:scorer_error"),    # agent
CLI 非零退出
    ("RateLimitError", "judge:api_error:rate_limit"), # litellm
异常类名
    ("ContentPolicyViolationError", "judge:api_error:content_filter"),
    ("AuthenticationError", "judge:api_error:auth"),
    ("litellm", "judge:api_error"),
)

```

```

def classify_exit(args, result):
    """归类为 AP 错误码，并把 Reward Kit 的原始错误文本原样透传给
exit_reason。

```

分类只做三件确定的事：命中源码里写死的异常消息 → 对应码；rewardkit 非零退出

→ scorer_error；跑通了没有任何有效条目 → invalid_output。其余 unknown。

无论哪种，exit_reason 都是 Reward Kit 自己的原话（error 字段或 stderr 末尾），

不做二次加工——排查时看到的应当是判分器实际报了什么。

```

"""
detail_msgs = detail_error_messages(pathlib.Path(args.graded))
stderr_tail = read_stderr_tail(pathlib.Path(args.graded))
haystack = " | ".join(detail_msgs) + "\n" + stderr_tail
reason = (detail_msgs[0] if detail_msgs else
          last_error_line(stderr_tail) or stderr_tail[-1000:]) or (
    "verifier marked unavailable without any error output")
for needle, code in _EXIT_CODE_RULES:
    if needle.lower() in haystack.lower():
        return code, reason
if args.graded_rc != 0:
    return "judge:scorer_error", reason
if not result.get("criteria_counted"):
    return "judge:invalid_output", reason
return "judge:unknown", reason

```

```

def compute(args):
    """汇总评分结果，返回要写进 reward.json 的字典。"""

```



```

graded_path = pathlib.Path(args.graded)

graded = load_scores(graded_path)
graded_details = load_json(graded_path.with_name("reward-
details.json"))

dims = {}
for key, value in (graded or {}).items():
    if key == "soft_score":
        continue
    number = finite(value)
    if number is not None:
        dims[key] = number

# 主分: 按签名权重池化全题 criterion (负向项真扣分, 空产物下限为 0)。
pooled, counted, broken = pooled_score(graded_details)
score = 0.0 if pooled is None else round(pooled, 6)

# Reward Kit 自己的 [0,1] 归一化聚合值, 仅留作审计参照, 不作主分。
soft = finite((graded or {}).get("soft_score"))
if soft is not None:
    soft = round(soft, 6)

graded_ok = (args.graded_rc == 0 and bool(dims) and pooled is not
None
              and counted > 0 and broken == 0)

result = dict(dims)
result["graded_score"] = score
result["criteria_counted"] = float(counted)
if soft is not None:
    result["soft_score"] = soft
# 评分不可用时主分一律记 0: 宁可保守低估, 也不要因为把异常条目排除在分母之
外
# 而把剩下的条目重新归一化成一个虚高的分数。真实分数留在 graded_score 里。
unavailable = not graded_ok
result["reward"] = 0.0 if unavailable else score
# 平台读取: 1 = 本次评分不可信 (判官限额/超时/评分器异常), 须重评而非记零
分。
result["verifier_error"] = 1.0 if unavailable else 0.0
return result

def main() -> int:

```

```

# 被 rewardkit discover() import 时不得有副作用：它会把 tests/ 下所有
*.py
# 都 import 一遍。若 argparse 留在模块级，import 即 SystemExit 杀死评分
进程。

parser = argparse.ArgumentParser()
parser.add_argument("--graded", required=True)
parser.add_argument("--graded-rc", type=int, required=True)
parser.add_argument("--out", required=True)
args = parser.parse_args()

try:
    result = compute(args)
except Exception:
    # 未预期的异常也必须落地一份结果：缺了 reward.json，平台读到的是"这道
    题没跑过"，
    # 与"跑出 0 分"无法区分。一律记 verifier_error = 1 交平台重评。
    result = {"graded_score": 0.0, "criteria_counted": 0.0,
              "reward": 0.0, "verifier_error": 1.0}

out_path = pathlib.Path(args.out)
out_path.parent.mkdir(parents=True, exist_ok=True)
out_path.write_text(json.dumps(result, ensure_ascii=False,
indent=2), encoding="utf-8")

# ---- AP 标准化输出 ----
# reward.txt: 单一数值（覆盖 test.sh 开头的 fail-closed 占位）。
out_path.with_name("reward.txt").write_text(
    f"{result.get('reward', 0.0)}\n", encoding="utf-8")
# reward_exit_message.json: 仅评分不可用时存在；成功则删除 fail-closed
占位。
exit_path = out_path.with_name("reward_exit_message.json")
if result.get("verifier_error"):
    try:
        code, reason = classify_exit(args, result)
    except Exception:
        code, reason = "judge:unknown", "classification itself
failed"
    exit_path.write_text(json.dumps({
        "exit_code": code,
        "exit_reason": reason,
        "extra_fields": {
            "criteria_counted": result.get("criteria_counted", 0.0),
            "graded_score": result.get("graded_score", 0.0),
            "rewardkit_rc": args.graded_rc,
        },
    },

```

```

    }, ensure_ascii=False, indent=2), encoding="utf-8")
else:
    exit_path.unlink(missing_ok=True)
# reward-details.json: 逐条明细随主分一并落到 out 同目录（rewardkit 写在
# --output 旁的 graded/ 子目录）。审计便利件，不参与 fail-closed 契约：
# 缺失或拷贝失败不影响主分与错误码归类，静默跳过。
graded_details = pathlib.Path(args.graded).with_name("reward-
details.json")
if graded_details.is_file():
    try:
        out_path.with_name("reward-details.json").write_text(
            graded_details.read_text(encoding="utf-8"),
encoding="utf-8")
    except OSError:

```